

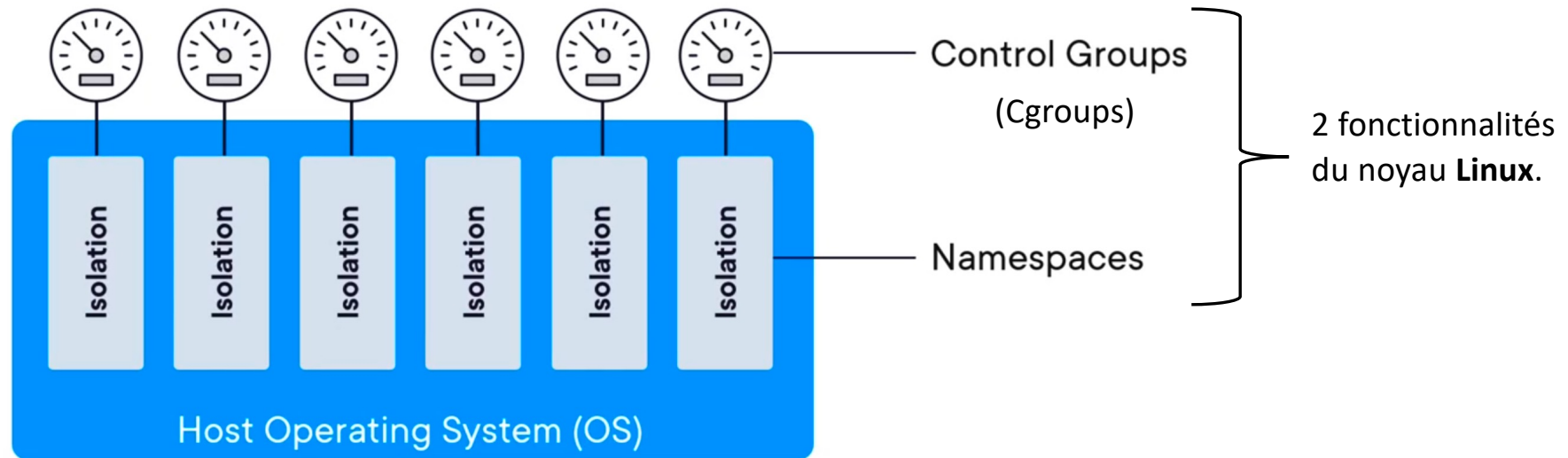


VIRTUALISATION PAR CONTENEURS

Conteneur

Container

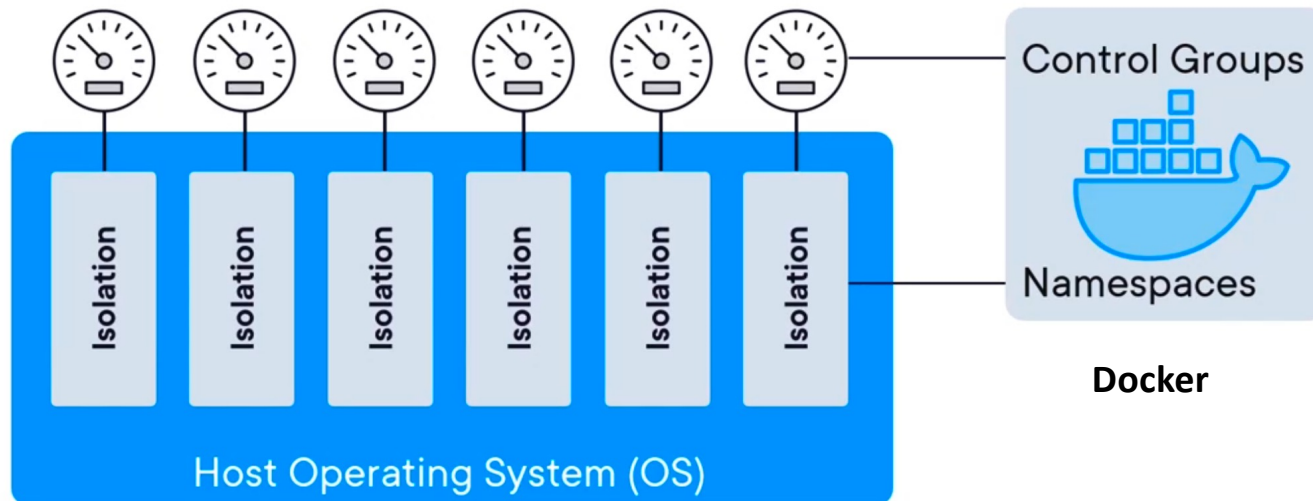
Isolated area of OS with resource limits applied.



Docker - Principe

Container

Isolated area of OS with resource limits applied.



Docker - Namespaces

Rôle des namespaces dans Docker

1.Isolation des ressources

- Chaque conteneur pense qu'il est seul sur la machine, car les namespaces limitent ce qu'il peut voir et utiliser.
- Cela permet d'avoir plusieurs conteneurs tournant côte à côte, sans interférence directe.

2.Sécurité et cloisonnement

- Un processus dans un conteneur n'a pas accès aux processus ou aux ressources d'un autre conteneur, ni à celles de l'hôte (sauf si explicitement partagé).

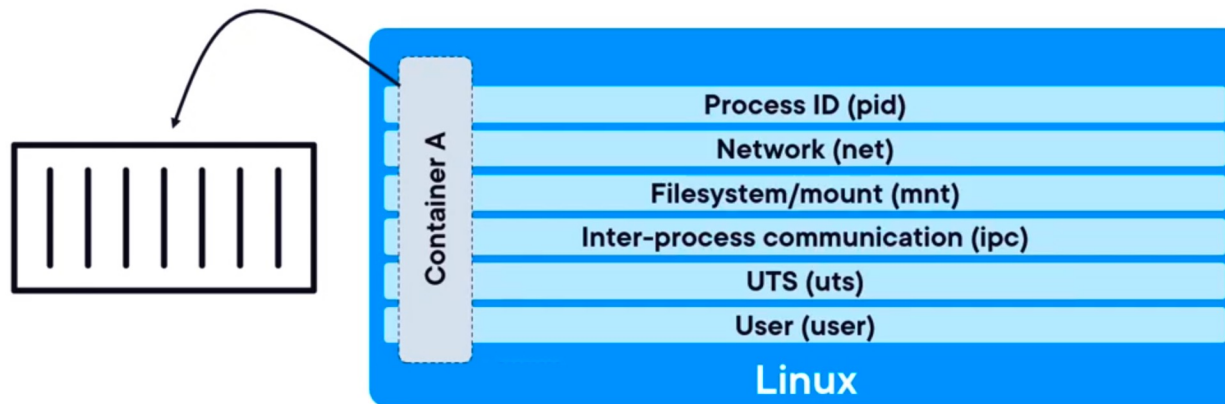
3.Flexibilité

- Docker combine plusieurs types de namespaces pour créer un environnement cohérent, proche d'une VM légère mais beaucoup plus rapide et consommant moins de ressources.

Docker - Types de namespaces

- **PID namespace :**
Isole les processus. Chaque conteneur a son propre arbre de processus (`ps` ne montre que les processus internes).
- **NET namespace :**
Isole la pile réseau (interfaces, routes, ports). Chaque conteneur peut avoir ses propres interfaces virtuelles, ponts, IP.
- **MNT (mount) namespace :**
Isole les points de montage et le système de fichiers. Permet à un conteneur d'avoir sa propre hiérarchie de répertoires, sans impacter l'hôte.
- **USER namespace :**
Isole les identités utilisateurs et groupes. À l'intérieur d'un conteneur, l'utilisateur peut sembler être `root`, mais être mappé à un utilisateur non privilégié sur l'hôte.

Docker - Namespaces



Namespaces
All about isolation

Control Groups
All about limits

Docker - Cgroups

1.Limiter les ressources utilisées par un conteneur

- CPU, mémoire, disque, réseau : chaque conteneur peut être plafonné.
- Exemple : empêcher un conteneur buggué de consommer toute la RAM de l'hôte.

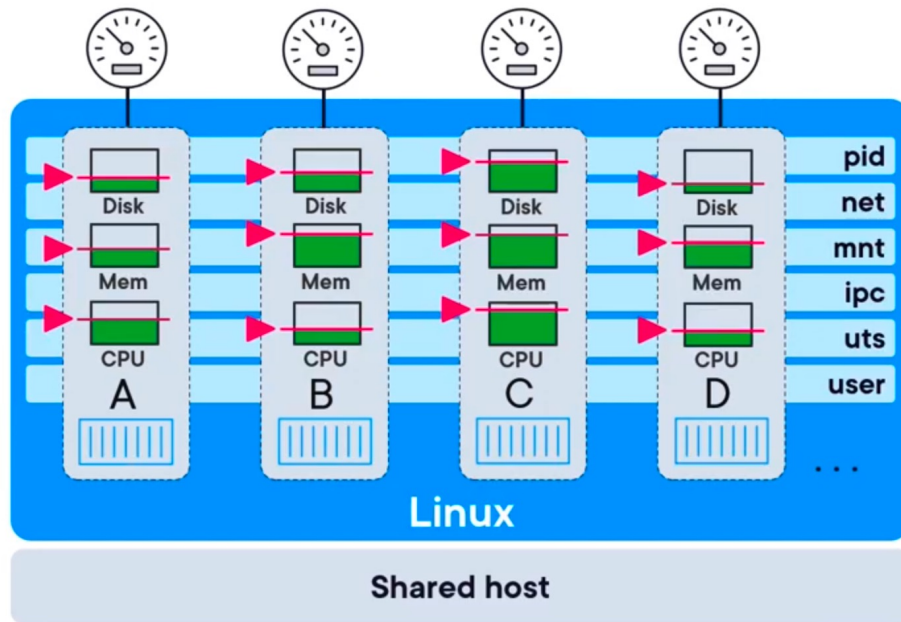
2.Garantir un partage équitable des ressources

- Plusieurs conteneurs peuvent cohabiter sur la même machine sans qu'un seul monopolise le CPU.
- Docker permet de définir des priorités (quotas, poids).

3.Surveiller la consommation

- Les cgroups permettent de suivre en temps réel combien de CPU, RAM, I/O sont consommés par un conteneur.
- Ces métriques sont utilisées par `docker stats` et par les orchestrateurs.

Docker - Principe



La combinaison des **namespaces** et des **Cgroups** permettent à Docker d'offrir une **virtualisation légère** :

- **Namespaces** donnent l'impression à chaque conteneur qu'il est seul.
- **Cgroups** garantissent que les ressources sont partagées équitablement et limitées.

Namespaces
All about isolation

+

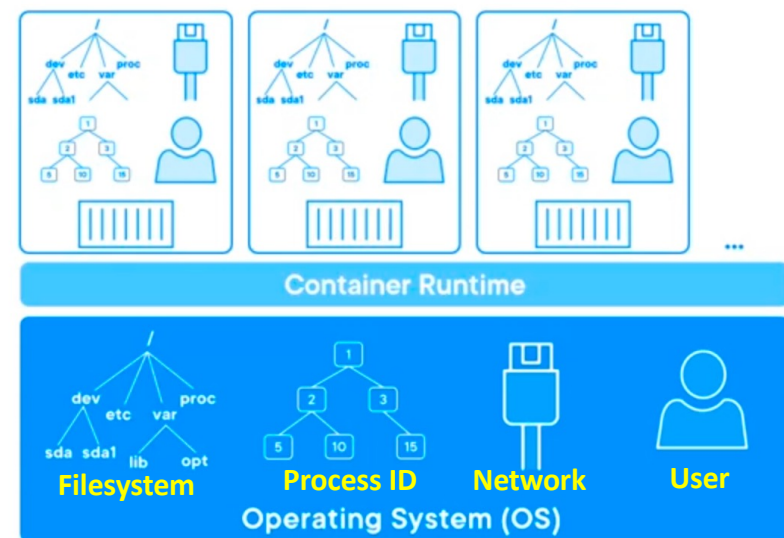
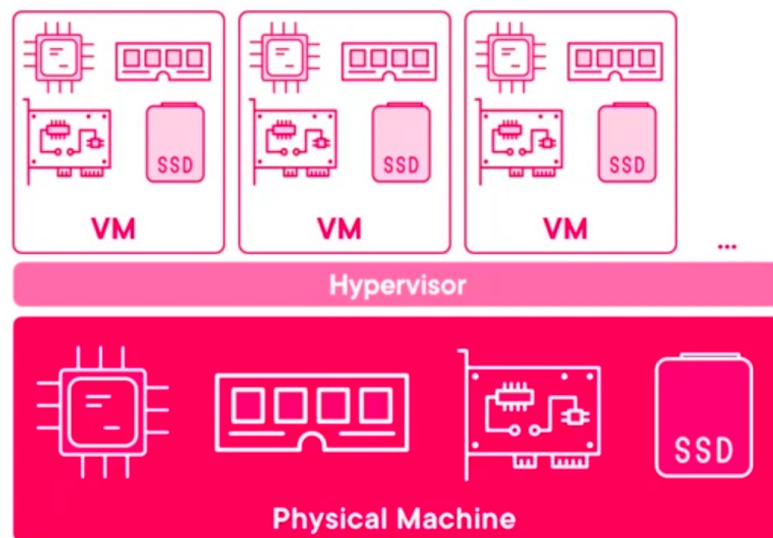
Control Groups
All about limits

=

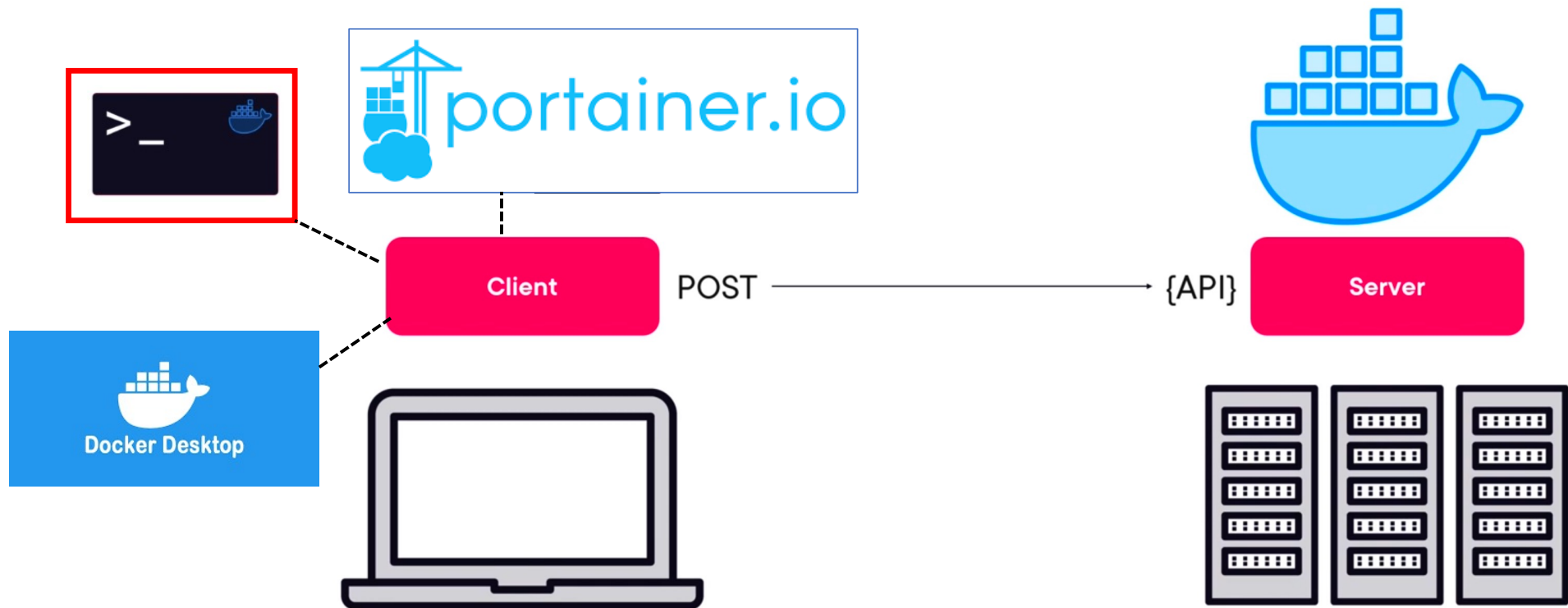


Conteneur Docker

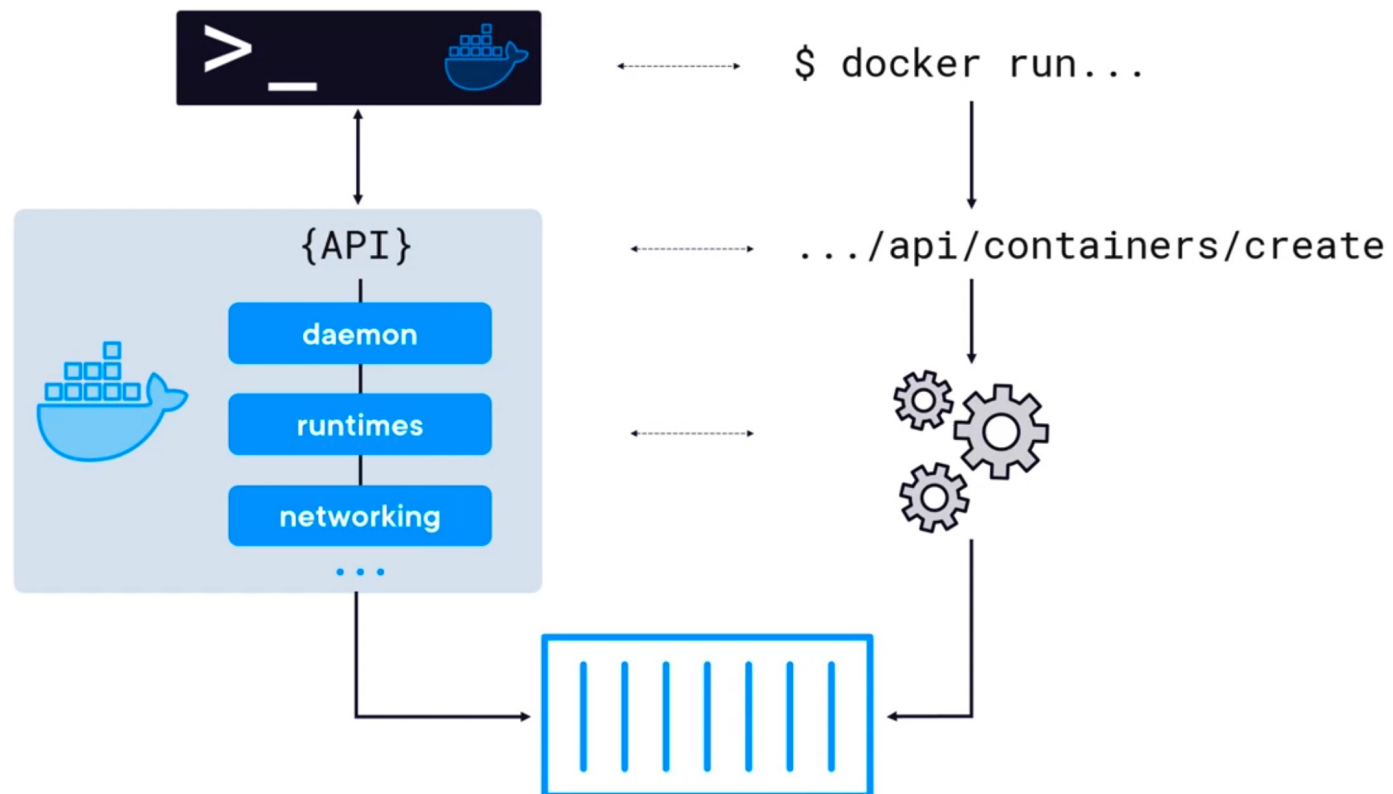
Docker – VM vs Conteneur



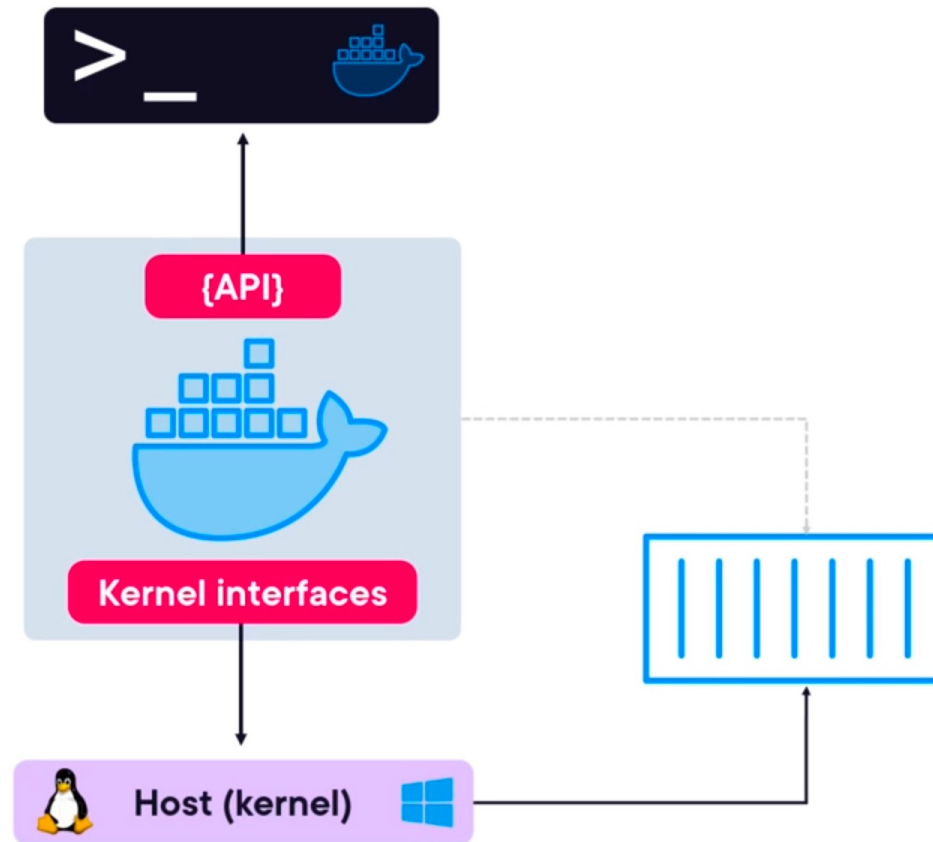
Docker – Architecture Client/serveur



Docker - Principe



Docker - Principe



Installer Docker

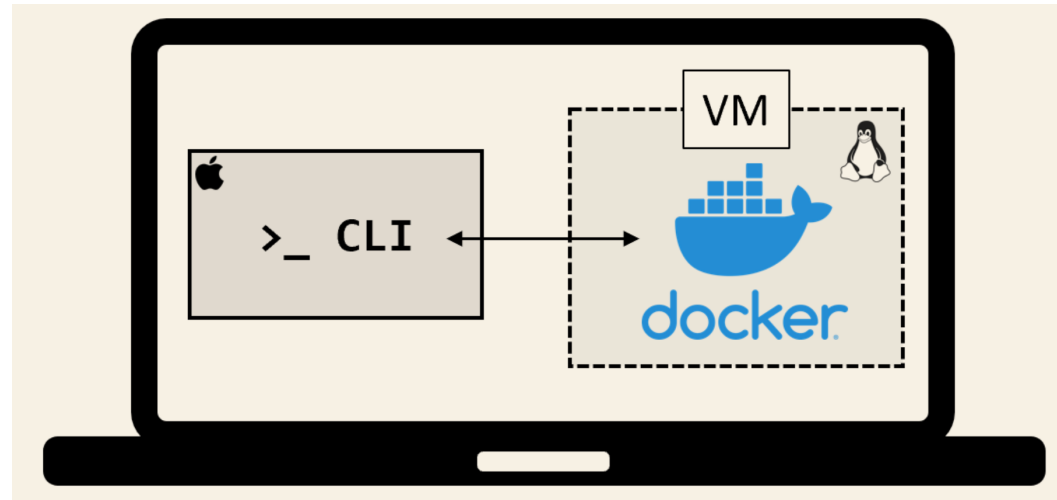
↓ Install Docker Desktop

Install Docker Desktop on [Mac](#),
[Windows](#), or [Linux](#).

```
> docker --version
Docker version 27.2.0, build 3ab4256

~
> docker -v
Docker version 27.2.0, build 3ab4256
```

<https://docs.docker.com/desktop/>



Architecture Docker

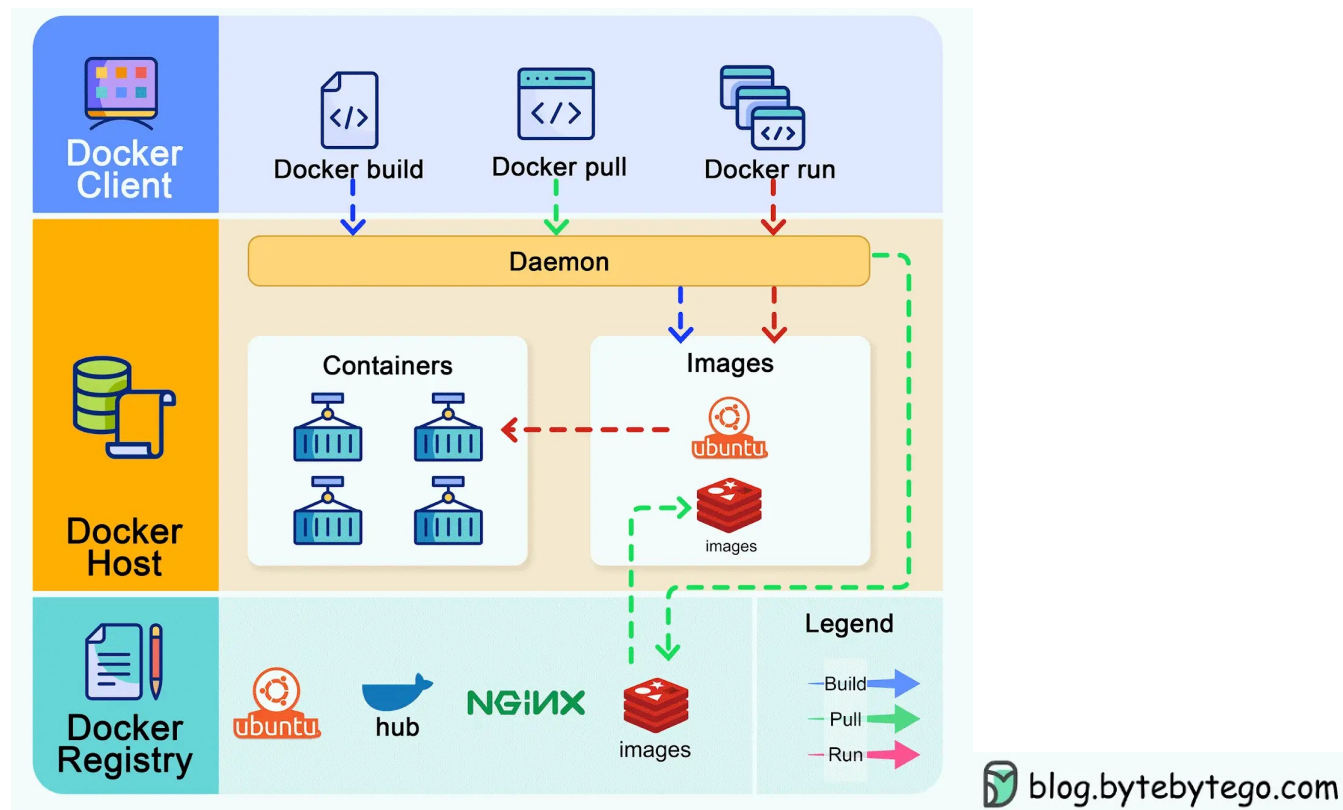


Image vs conteneur

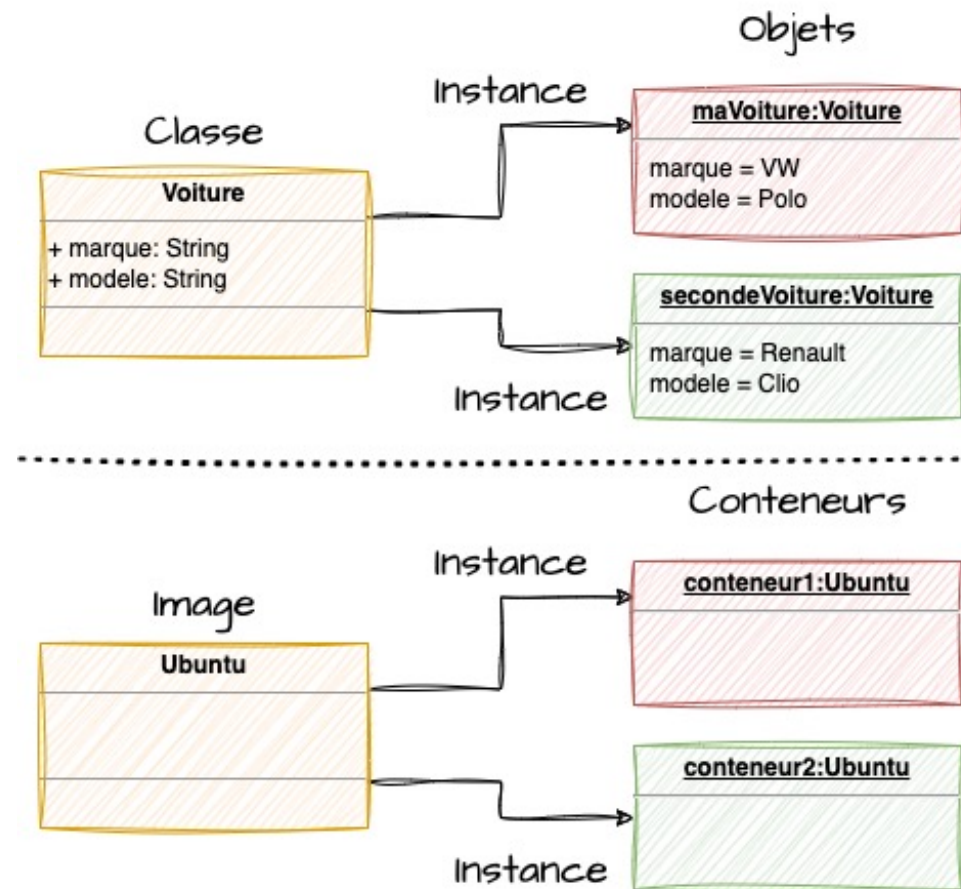


Image vs conteneur

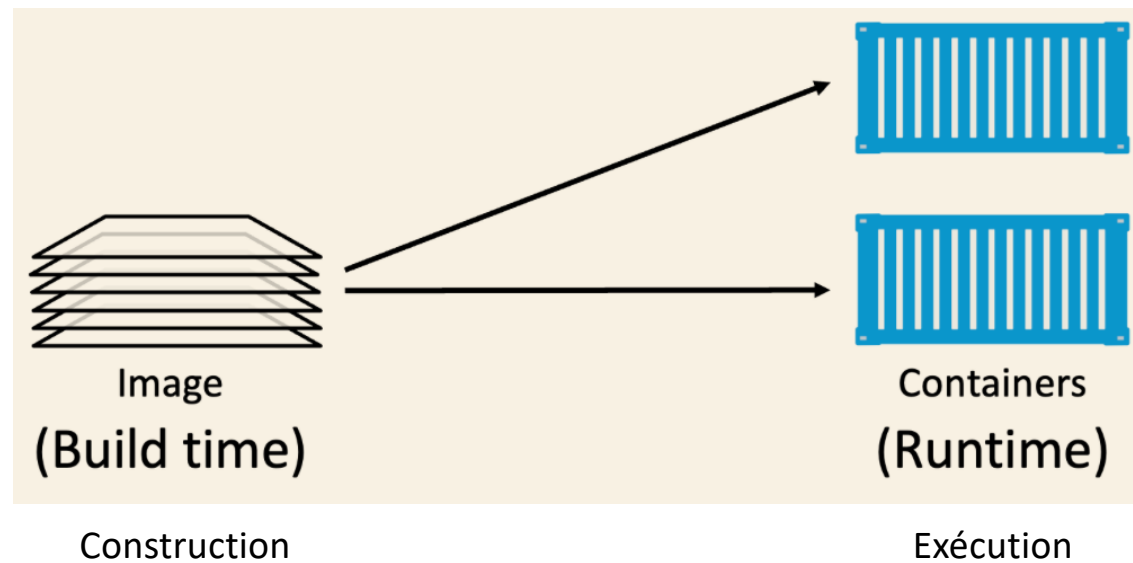


Image vs conteneur



All mean the same

Image

Container image

Docker Image

OCI image



All mean the same

Container

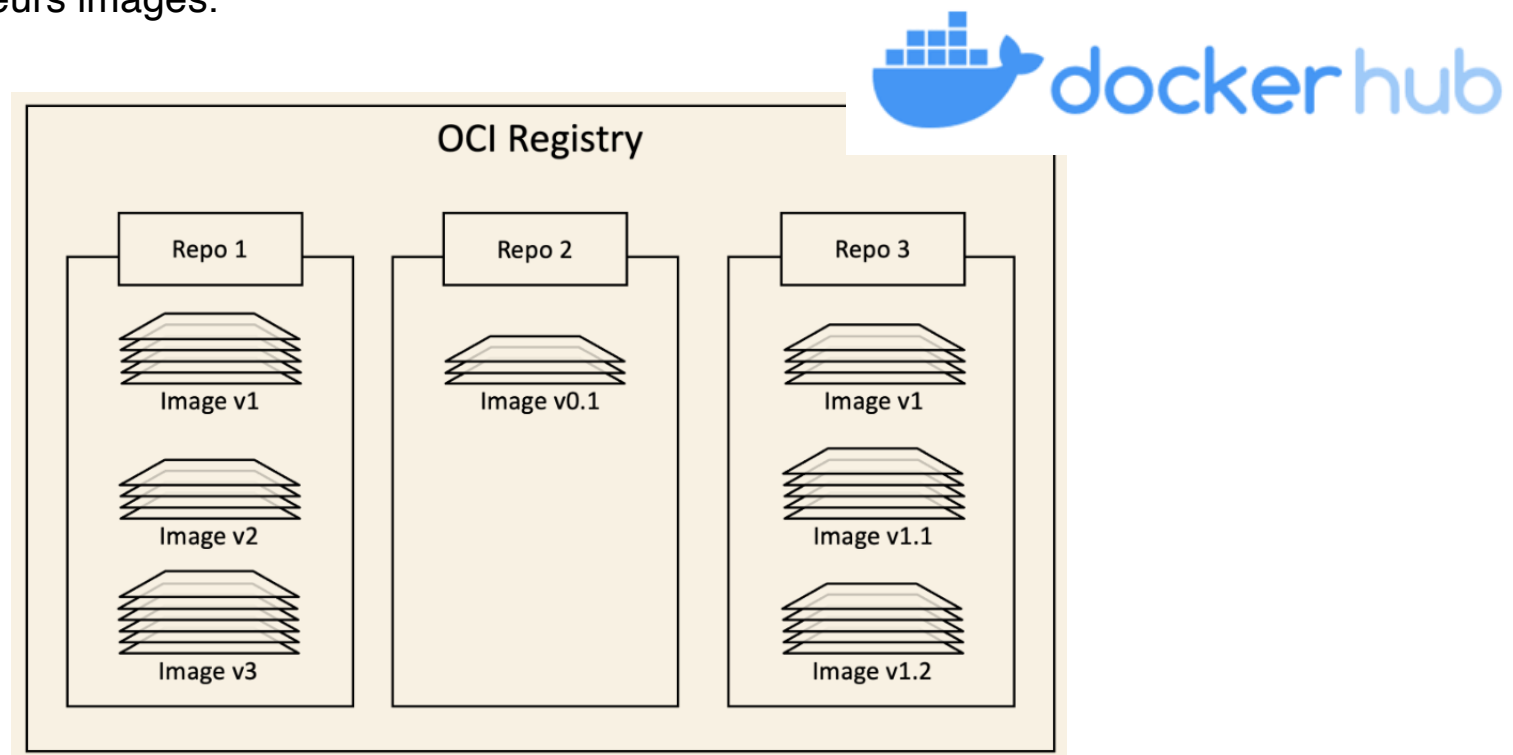
Docker container

OCI container

<https://www.pluralsight.com/courses/docker-deep-dive-2023>

Où trouve-t-on des images ?

Un registre Docker contient un ou plusieurs dépôts (repository) d'images. Un dépôt renferme une ou plusieurs images.



Comment récupérer une image en local?

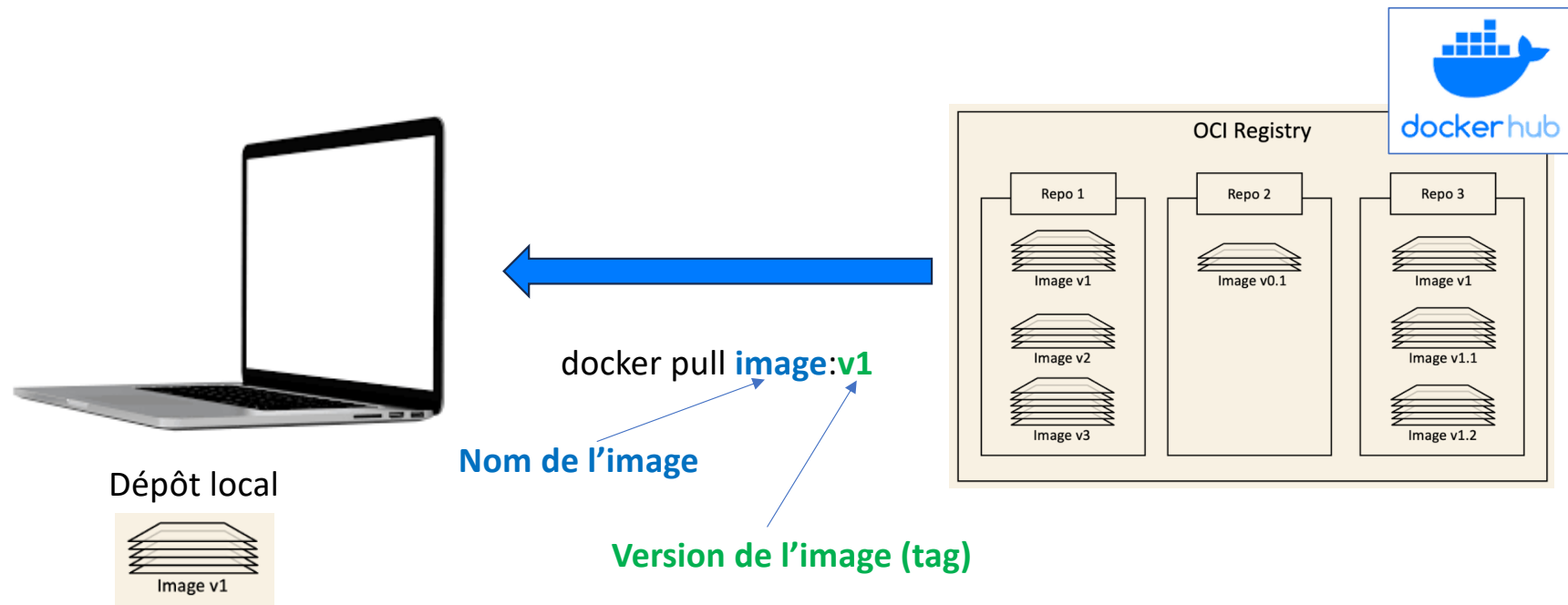
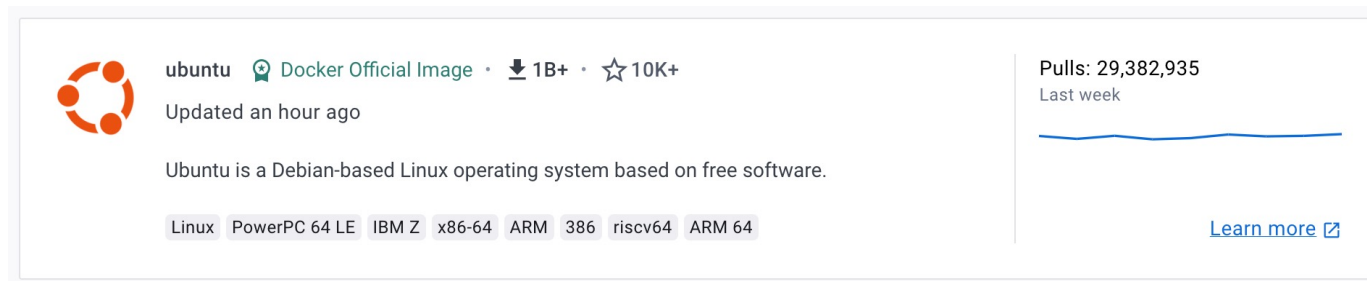


Image Docker



The screenshot shows the Docker Hub interface for the 'ubuntu' Docker Official Image. On the left, there is the Ubuntu logo (four orange circles) and the text 'ubuntu Docker Official Image' with a green verified badge. Below this, it says 'Updated an hour ago' and 'Ubuntu is a Debian-based Linux operating system based on free software.' At the bottom, there are tags for various architectures: Linux, PowerPC 64 LE, IBM Z, x86-64, ARM, 386, riscv64, and ARM 64. On the right, it displays 'Pulls: 29,382,935' and 'Last week' with a line graph showing a steady increase. A 'Learn more' link with an external icon is at the bottom right.

ubuntu  Docker Official Image ·  1B+ ·  10K+

Updated an hour ago

Ubuntu is a Debian-based Linux operating system based on free software.

Linux PowerPC 64 LE IBM Z x86-64 ARM 386 riscv64 ARM 64

Pulls: 29,382,935
Last week

[Learn more](#) 

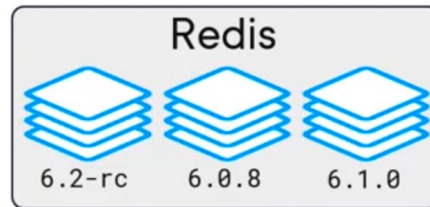
Image Docker Officielle : Elle doit contenir du code :

- à jour
- de haute qualité
- Sécurisé
- bien documenté
- conforme aux meilleures pratiques.

Registre – Dépôt officiel

Official Repos

OCI Registry



E.g. `docker.io/redis:6.0.8`

```
$ Docker pull redis:6.0.8
```

Registre – dépôt non officiel

Unofficial Repos

User/org

```
$ docker pull bitnami/redis:6.0.8
```

Afficher le contenu du dépôt d'images en local

```
(base) giannitricarico@MacBook-Pro-de-Gianni ~ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
ecommerceapi	latest	4594e6458c5a	6 months ago	621MB
nginx	latest	b21c88e97517	9 months ago	142MB
backup	latest	8c3550ee80a8	9 months ago	4.03MB
mongo	latest	0850fead9327	9 months ago	700MB

Images

```
(base) giannitricarico@MacBook-Pro-de-Gianni ~ > docker pull nginx:latest
latest: Pulling from library/nginx
a803e7c4b030: Pull complete
8b625c47d697: Pull complete
4d3239651a63: Pull complete
0f816efa513d: Pull complete
01d159b8db2f: Pull complete
5fb9a81470f3: Pull complete
9b1e1e7164db: Pull complete
Digest: sha256:32da30332506740a2f7c34d5dc70467b7f14ec67d912703568daff790ab3f755
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
```

L'**OverlayFS** est un **système de fichiers unifié** du noyau Linux. Son rôle principal est de permettre d'**empiler plusieurs systèmes de fichiers** (en couches) et de présenter à l'utilisateur une **vue unique**.



Image Docker = pile de couches lecture seule

Images – utilité d'OverlayFS

1. Images Docker construites en couches

- OverlayFS permet d'**empiler ces couches** les unes sur les autres, comme des calques.

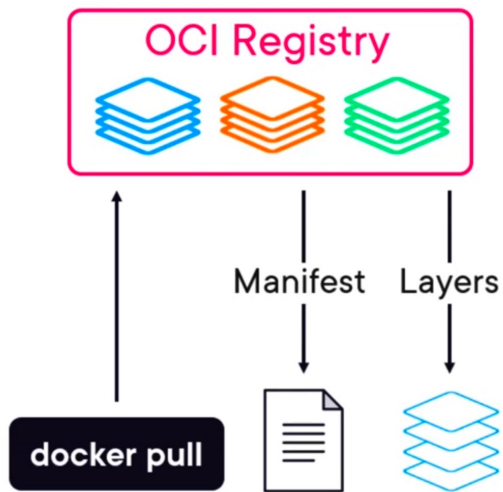
2. Espace disque optimisé

- Les couches sont **réutilisées** entre conteneurs/images.
- Exemple : si 10 conteneurs utilisent la même base `ubuntu:20.04`, cette couche est stockée une seule fois.

3. Isolation des conteneurs

1. Chaque conteneur a sa **couche supérieure en écriture**, indépendante.
2. Ainsi, modifier un fichier dans un conteneur n'affecte pas l'image de base ni les autres conteneurs.

Images



Le **fichier manifest** est la “carte d’identité” de l’image Docker :

- Il décrit son contenu (couches, config).
- Il garantit l’intégrité (hash SHA256) et évite les téléchargements redondants.

Sans le **manifest**, Docker ne saurait pas quelles couches récupérer ni comment assembler une image correcte.

Images

`docker inspect nginx:latest`

```
    "RootFS": {  
      "Type": "layers",  
      "Layers": [  
        "sha256:d310e774110ab038b30c6a5f7b7f7dd527dbe527854496bd30194b9ee6ea496e",  
        "sha256:eb7e3384f0ab3a08dd0163ab66625d954e511e8fa112c09923e261418c8524cb",  
        "sha256:1dc45c680d0f1615d45021e85028c5d2089468518ec44b7aeeb947cdf0b22364",  
        "sha256:ea43d4f82a03877421d6bed1db76fd58f87555193a7921cefb22d6067d8963a7",  
        "sha256:9c6261b5d19808f0feab896f5f65c0f6287aa8589f880d4af64414608e54c023",  
        "sha256:a7e2a768c198ee94c3a3e8a1cc199a69510da1e39fd85beddc2d4f3200a21d1e",  
        "sha256:d26d4f0eb474290793ba42d20b4880daa7c6ddf0379adeb1c121e04f2840712e"  
      ]  
    },  
  ],  
}
```

Couches + manifest = image

Images – Commande

Suppression d'une image

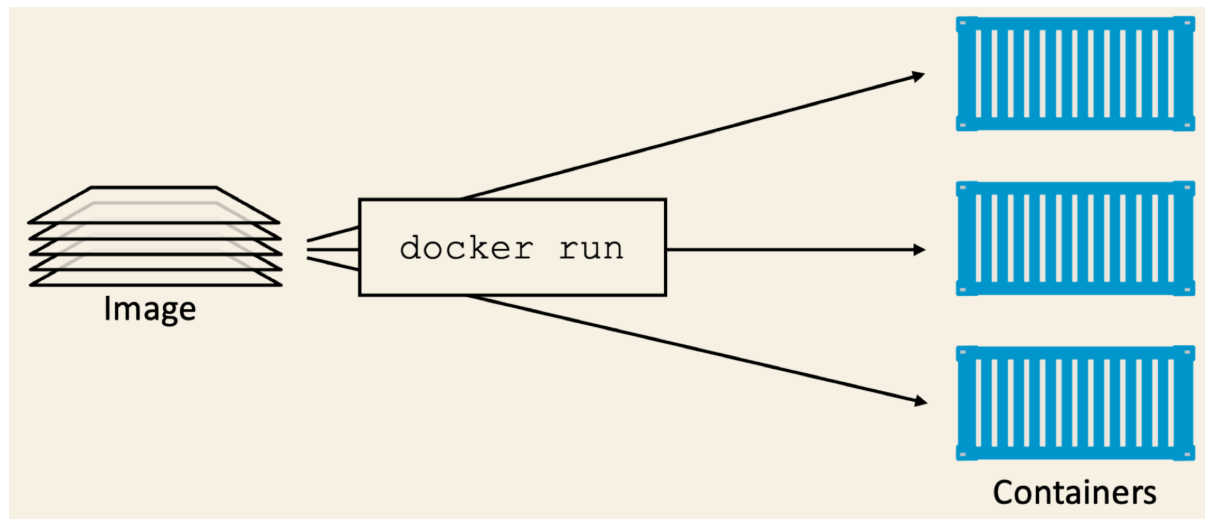
```
$ docker rmi [ID de l'image]
```


Gestion des images – Exercice

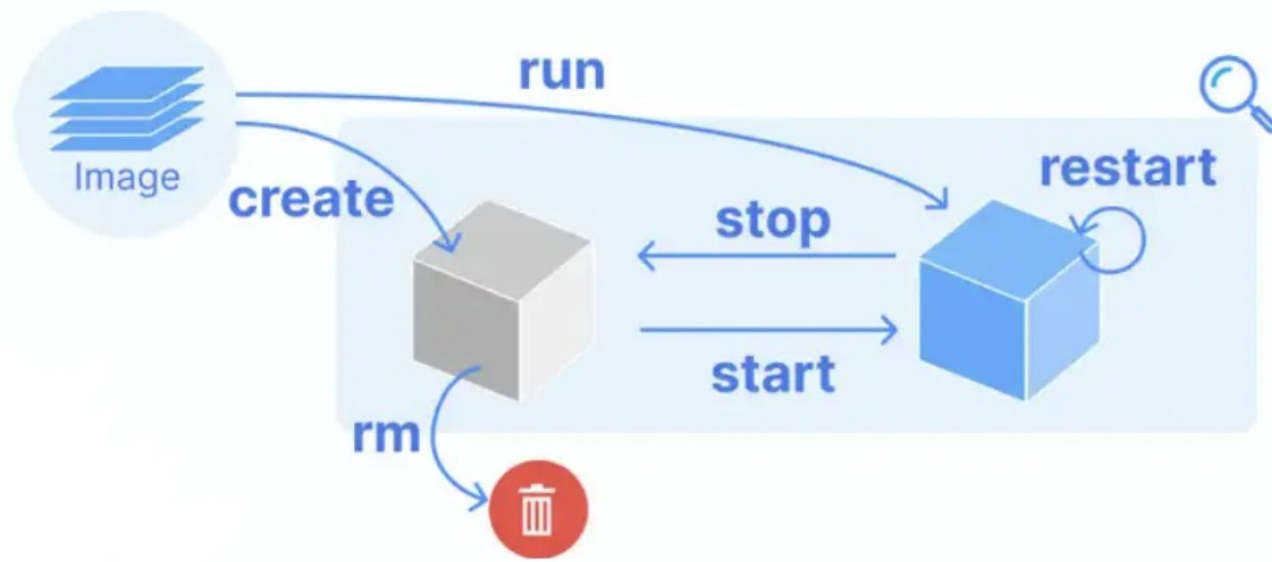
L'objectif de cet exercice est de vous familiariser avec le téléchargement et la suppression d'images. Dans cet exercice, vous vous concentrerez sur deux applications populaires : **Memcached** (système de cache distribué en mémoire) et **Nginx**(serveur web).

- 1) Télécharger les images
- 2) Vérifiez que les images sont présentes
- 3) Supprimer les images
- 4) Vérifiez que les images sont supprimées.

Conteneur – Création et exécution



Conteneur – Cycle de vie d'un conteneur



Conteneur - Options

```
$ docker run [OPTIONS] IMAGE:TAG [COMMAND] [ARG...]
```

=> **crée** un conteneur et le **démarre** en une seule opération

Options

-it : permet d'ouvrir un pseudo terminal sur le conteneur -> mode interactif.

--name : permet de nommer le conteneur.

--rm : supprime automatiquement un conteneur à l'arrêt.

Conteneur – Commandes systèmes

- `$ docker ps`
 - affiche les conteneurs en cours d'exécution
- `$ docker ps -a`
 - affiche tous les conteneurs (à l'arrêt et à l'exécution)
- `$ docker inspect`
 - affiche toutes les informations liées au conteneur

Conteneur – OS Linux

```
$ docker run [OPTIONS] IMAGE:TAG [COMMAND] [ARG...]
```

Exemples :

```
$ docker run alpine:latest sleep 5
```

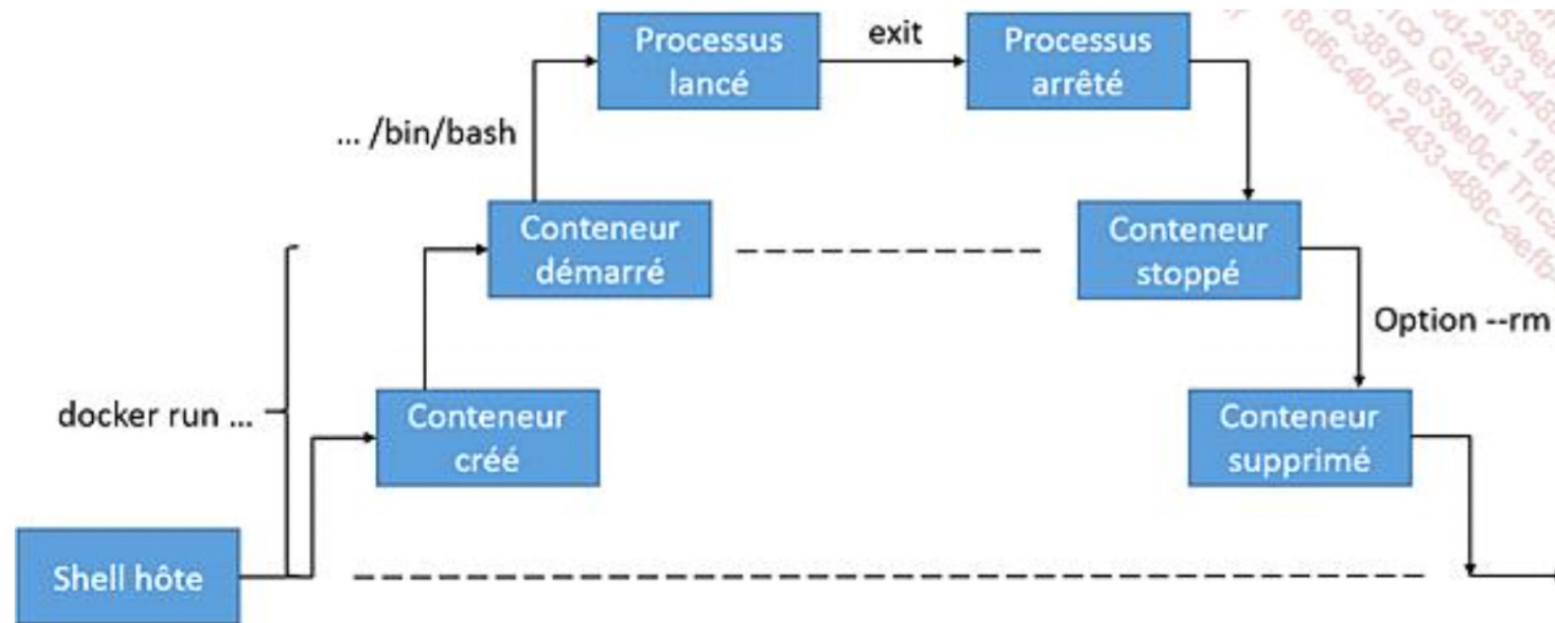
```
$ docker run --rm -it alpine sh
```

```
/ # ls
bin    dev    etc    home   lib    media  mnt    opt    proc   root   run    sbin   srv    sys    tmp    usr    var
/ # exit
```

```
bin    dev    etc    home   lib    media  mnt    opt    proc   root   run    sbin   srv    sys    tmp    usr    var
/ # exit
~ took 4m35s
> █
```

Conteneur – Serveur web

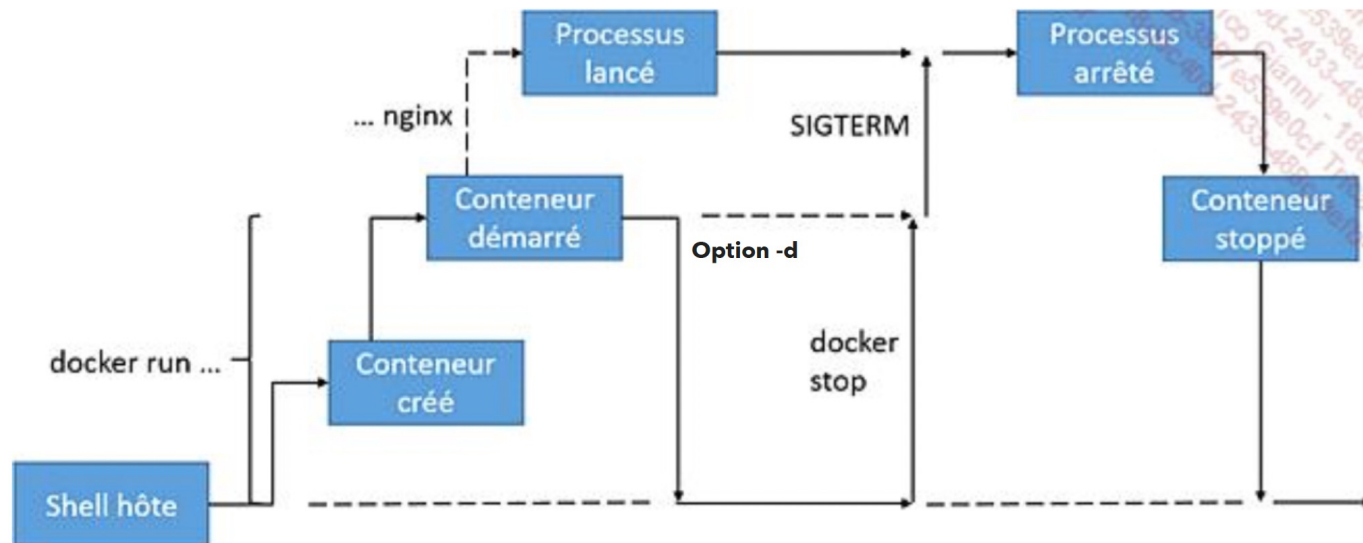
Mode bloquant (hôte)



```
$ docker run nginx
```

Conteneur – Serveur web

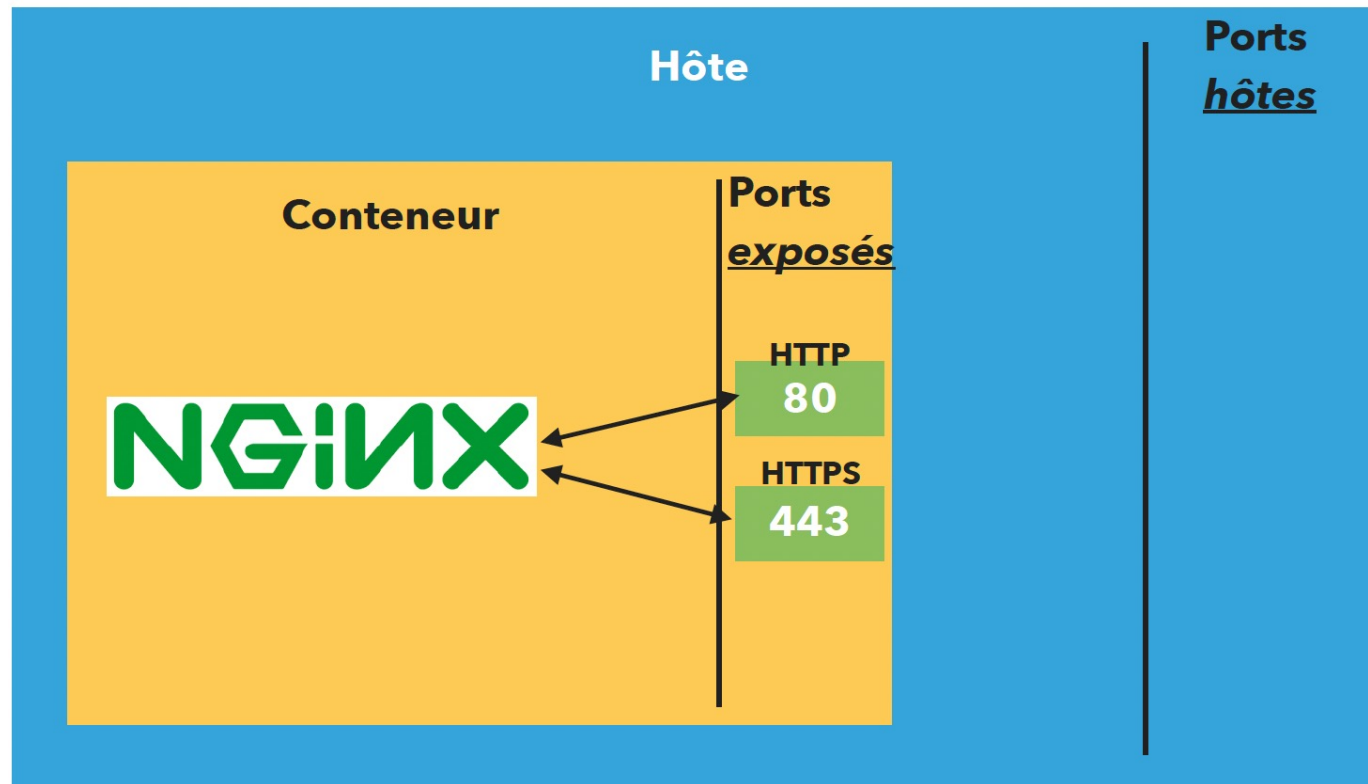
Mode non bloquant (hôte)



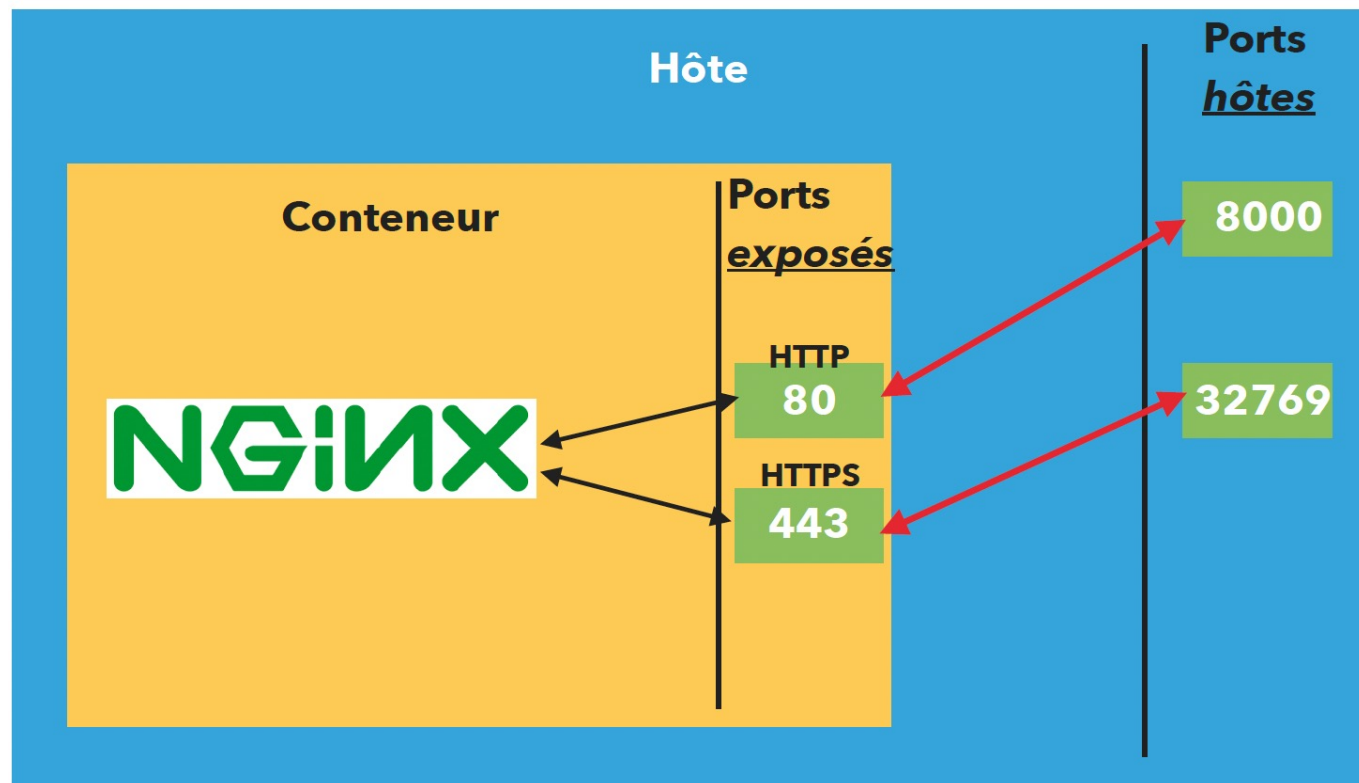
```
$ docker run -d nginx
```

- d : processus daemon (arrière-plan) et détaché du terminal local

Conteneur – Serveur web – ports exposés



Conteneur – Serveur web – Mappage



Conteneur – Serveur web – Mappage de ports

Mappage automatique

- Mapper automatiquement tous les ports exposés dans le conteneur.

docker run -P -d nginx

Mappage manuel

- spécifier manuellement quels ports doivent être mappés parmi ceux exposés
- spécifier manuellement les ports hôtes à utiliser

Option **-p** de la commande docker run

- Sans le port hôte : **-p 80** (mappage automatique)
- Avec le port hôte : **-p 8000:80** (mappage manuel) => -p **port hôte**: **port conteneur**

```
docker run -d --name webServer2 -p 8000:80 nginx
```

Conteneur – Serveur web

```
docker exec -it nomConteneur /bin/bash
```

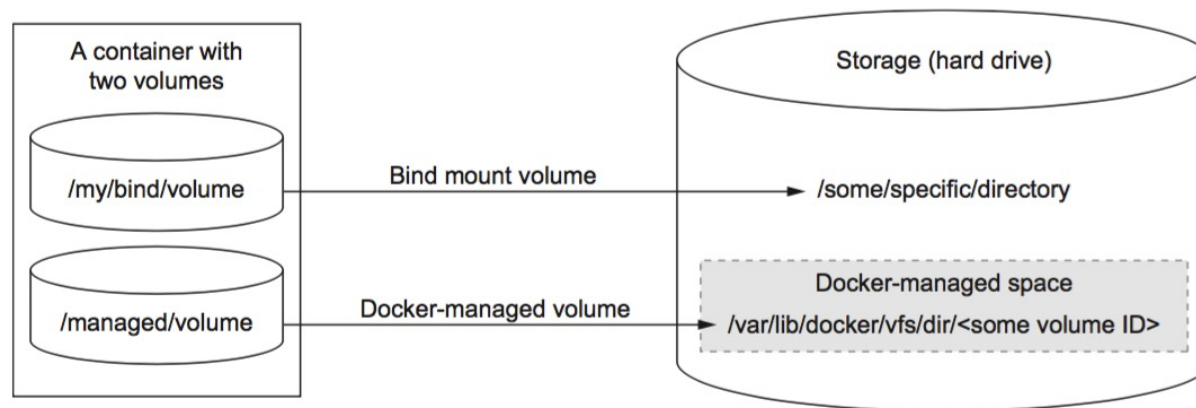
=> Permits de lancer un terminal bash dans un conteneur en cours d'exécution.

```
/usr/share/nginx/html
```

Conteneur – Volumes

- Type de **dossier spécial** dans un conteneur
- Dossier qui peut être **partagé**
- Volumes permettent de gérer la **persistance** des données même après la suppression du conteneur.

Conteneur – Volumes



Conteneur – Volumes

-v <dossier hôte> : <dossier conteneur>

```
Docker run -d -P -v /Users/gianni/Documents/Labs/WebDocker  
:/usr/share/nginx/html nginx
```

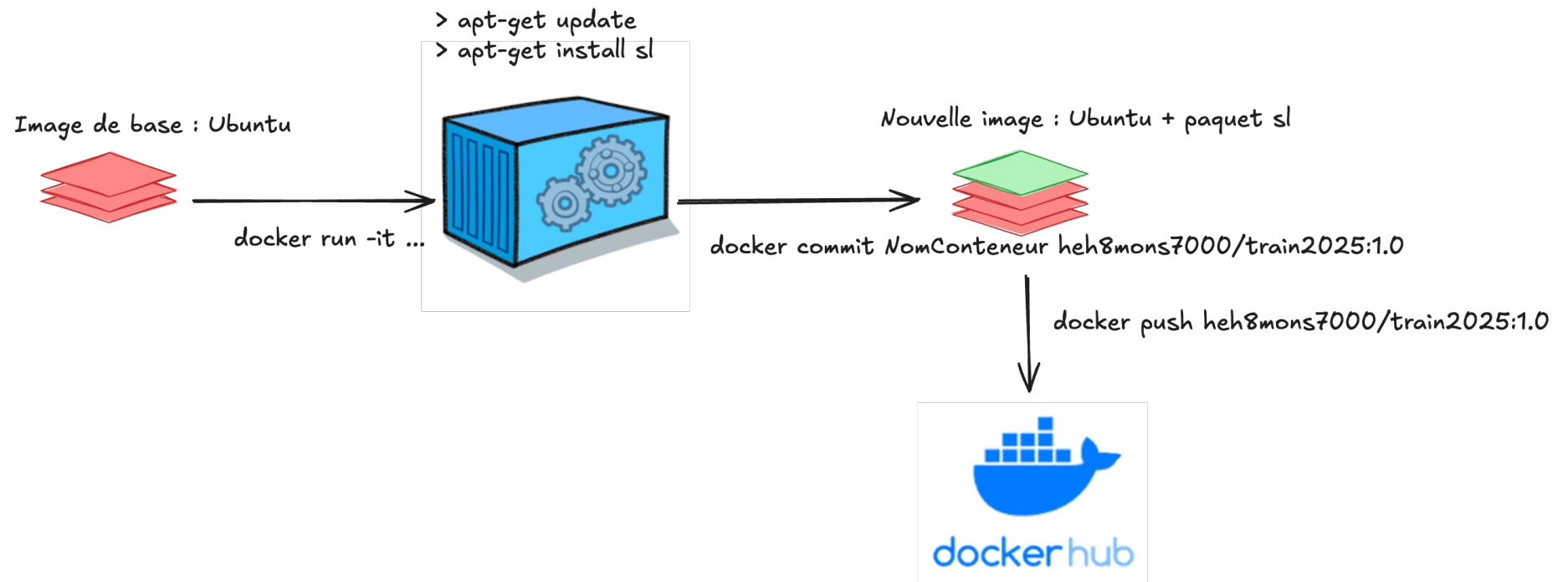
Conteneur serveur DB – Variables d'environnement

Passer des variables d'environnement à un conteneur.

```
docker run -p 3306:3306 --name mysqlDB -e MYSQL_ROOT_PASSWORD=root  
-d mysql:latest
```


Construction d'images – Conteneur

1) Création d'une image à partir d'un conteneur



Construction d'images – Conteneur

2) Logger sur Docker Hub

```
> docker login
```

USING WEB BASED LOGIN

To sign in with credentials on the command line

Your one-time device confirmation code is: **HDNL**

Press ENTER to open your browser or submit your

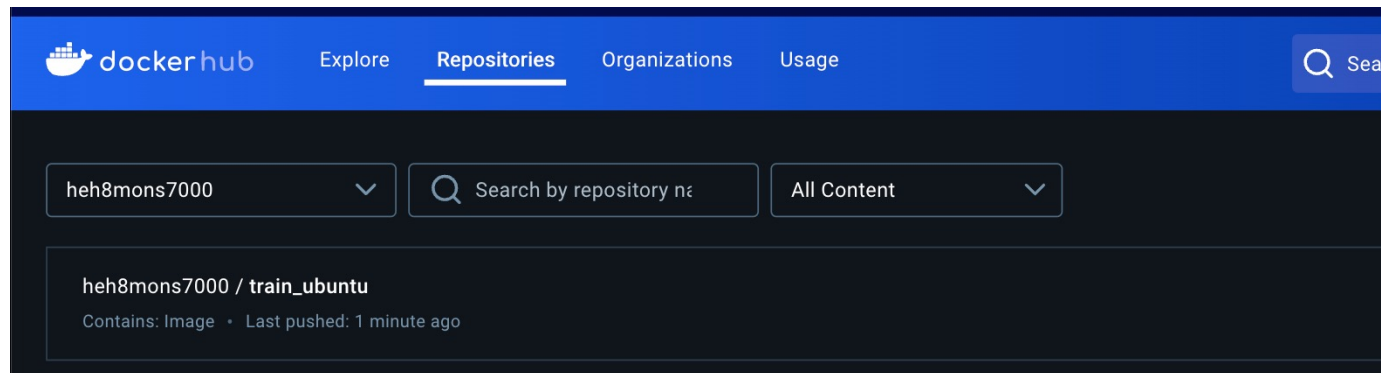
Waiting for authentication in the browser...

Login Succeeded

Construction d'images – Conteneur

3) Publier son image sur Docker Hub

```
> docker push heh8mons7000/train_ubuntu:1.0
The push refers to repository [docker.io/heh8mons7000/train_ubuntu]
f73d1a40c796: Pushed
a4feb6502043: Mounted from library/ubuntu
1.0: digest: sha256:0eeca2662399ebaa43fa74470c37f80aaf52b24749d1a05791cf5808aed07c4a size: 741
```



Construction d'images – Conteneur

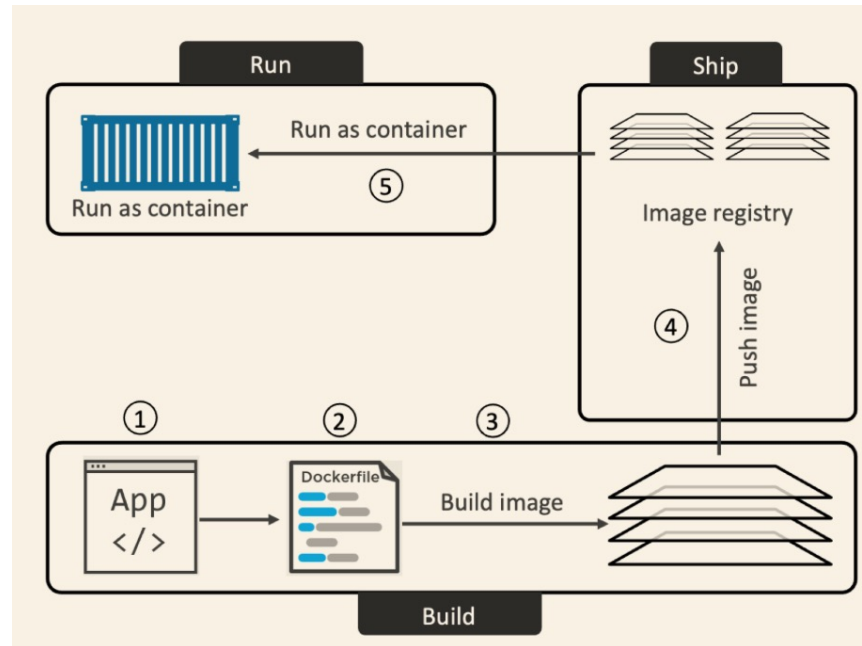
```
docker run -t --rm heh8mons7000/train2025:1.0 /usr/games/sl
```

```
docker run -t --rm heh8mons7000/train2025:1.0 /usr/games/sl -F
```

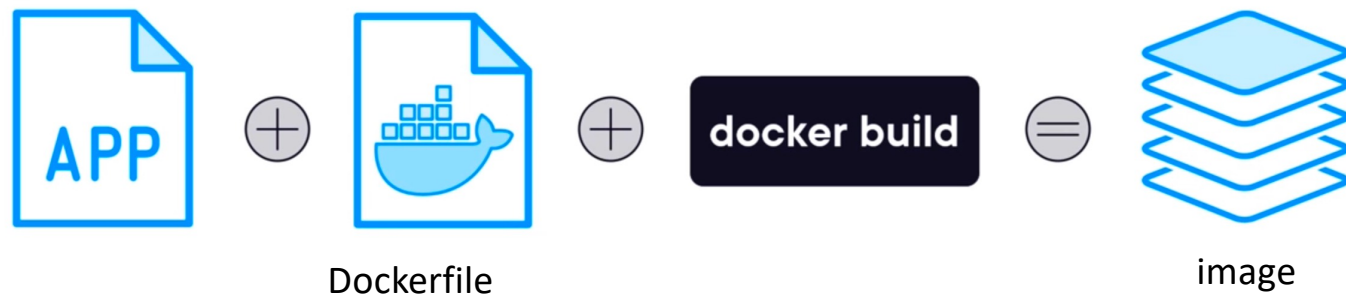
Avec option : -F (Fly)

Construction d'images – Dockerfile

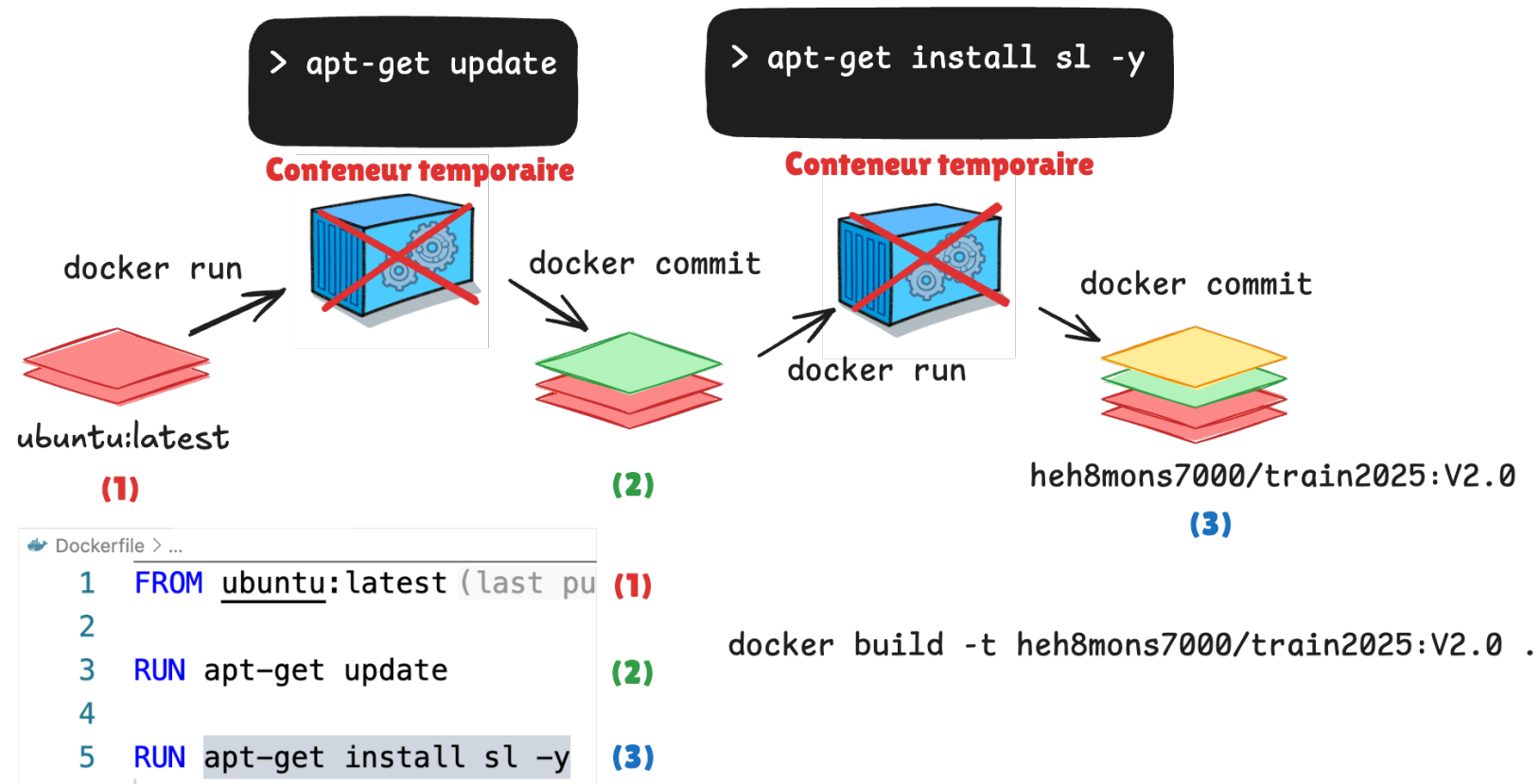
Création d'une image à partir d'un fichier Dockerfile



Construction d'images – Dockerfile



Construction d'images – Dockerfile



Construction d'images – Dockerfile

```
=> [1/3] FROM docker.io/library/ubuntu:24.10  
=> [2/3] RUN apt-get update  
=> [3/3] RUN apt-get install sl -y  
=> exporting to image
```

```
=> [1/3] FROM docker.io/library/ubuntu:24.10  
=> CACHED [2/3] RUN apt-get update  
=> [3/3] RUN apt-get install sl cowsay -y  
=> exporting to image
```

Ajout d'un paquet => pas de mise à jour des dépendances !

Construction d'images – Dockerfile

 Dockerfile > ...

```
1 FROM ubuntu:latest (last pushed 1 week ago)
2
3 RUN apt-get update && apt-get install sl cowsay nano -y
```

Documents/labDocker/Exemples via  desktop-linux took 13s

> docker build -t heh8mons7000/train2025:V2.3 .

[+] Building 12.5s (6/6) FINISHED

=> [internal] load build definition from Dockerfile	docker:desktop-linux
=> => transferring dockerfile: 116B	0.0s
=> [internal] load metadata for docker.io/library/ubuntu:latest	0.0s
=> [internal] load .dockerignore	0.0s
=> => transferring context: 2B	0.0s
=> CACHED [1/2] FROM docker.io/library/ubuntu:latest	0.0s
=> [2/2] RUN apt-get update && apt-get install sl cowsay nano -y	12.3s
=> exporting to image	0.2s
=> => exporting layers	0.2s
=> => writing image sha256:d623117dffe07656223484e9aff64a6f30f695b93102a07793a4f859a3	0.0s
=> => naming to docker.io/heh8mons7000/train2025:V2.3	0.0s

Construction d'images – Dockerfile

```
=> => transferring context: 2B
=> CACHED [1/2] FROM docker.io/library/ubuntu:24.10
=> [2/2] RUN apt-get update && apt-get install sl cowsay -y
=> exporting to image
=> => exporting layers
```

```
=> => transferring context: 2B
=> CACHED [1/2] FROM docker.io/library/ubuntu:24.10
=> [2/2] RUN apt-get update && apt-get install sl cowsay nano -y
=> exporting to image
```

Construction d'images – Dockerfile

 Dockerfile > ...

```
1 FROM ubuntu:latest (last pushed 1 week ago)
2
3 RUN apt-get update && apt-get install sl cowsay nano -y
4
5 ENTRYPOINT [ "/usr/games/sl" ,"-l"]
```

Construction d'images – Dockerfile

 Dockerfile > ...

```
1 FROM ubuntu:latest (last pushed 1 week ago)
2
3 RUN apt-get update && apt-get install sl cowsay nano -y
4 WORKDIR /usr/games
5
6 ENTRYPOINT [ "./sl" , "-l"]
```

Construction d'images – Dockerfile

 Dockerfile > ...

```
1 FROM ubuntu:latest (last pushed 1 week ago)
2
3 RUN apt-get update && apt-get install sl cowsay nano -y
4
5 WORKDIR /usr/games
6
7 COPY script3trains.sh .
8 RUN chmod +x script3trains.sh
9
10 ENTRYPOINT [ "./script3trains.sh" ]
```

Construction d'images – Dockerfile

```
$ script3trains.sh
1  #!/bin/bash
2  for i in `seq 1 3`;
3  do
4      /usr/games/sl
5  done
```

Construction d'images – Dockerfile

 Dockerfile > ...

```
1 FROM ubuntu:latest (last pushed 1 week ago)
2
3 RUN apt-get update && apt-get install sl cowsay nano -y
4
5 WORKDIR /usr/games
6
7 COPY script3trains.sh .
8 RUN chmod +x script3trains.sh
9
10 ENV N=2
11
12 ENTRYPOINT [ "./script3trains.sh" ]
```

Construction d'images – Dockerfile

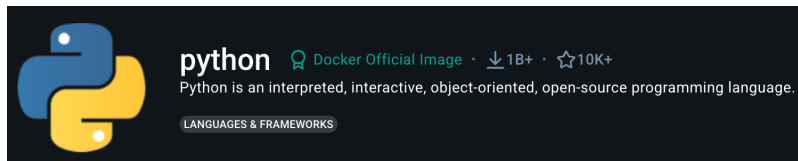
```
$ script3trains.sh
```

```
1  #!/bin/bash
2  for i in `seq 1 $N`;
3  do
4      /usr/games/sl
5  done
```


Construction d'images – Exercice

Conteneurisation d'une application web

Image de base



Flask

Flask est un petit framework web Python léger, qui fournit des outils et des fonctionnalités utiles qui facilitent la création d'applications web en Python.

Nom de l'image => app_web:1.0

Construction d'images – Déploiement d'un conteneur



Service cloud pour le déploiement => <https://render.com/>

Create a new **Web Service**

Connect a Git repository, or use an existing image.

How would you like to deploy your web service?

☒ Build and deploy from a Git repository
Connect a GitHub or GitLab repository.

☐ Deploy an existing image from a registry **ADVANCED**
Pull a public image from any registry or a private image from Docker Hub, GitHub, or GitLab.

Next

Construction d'images – Déploiement

Create a new **Web Service**

Deploy your web service from an image hosted on a public or private registry.

Deploy an image

Image URL

The image URL for your external image.



docker.io/heh8mons7000/app1:12



Credential (Optional)

Use a credential to access private images.
Manage credentials in [Settings](#).

No credential



Next

Registry Credentials

We currently support [private images](#) from
GitHub, GitLab, and Docker Hub registries.
We don't restrict public image registry types.

This is a public image that can be accessed
without credentials.

Construction d'images – Déploiement

Name
A unique name for your web service.

api_rest_test

Region
The [region](#) where your web service runs.

Frankfurt (EU Central) ▼

Please [enter your payment information](#) to select an instance type with higher limits.

Instance Type	RAM	CPU	Price
☒ Free	512 MB	0.1 CPU	\$0 / month

Construction d'images – Déploiement

WEB SERVICE

api_rest_test

Image Free

Connect

Manual Deploy

heh8mons7000 / app1

12

<https://api-rest-test-m6t3.onrender.com>

Events

Free instance types will spin down with inactivity. [Upgrade to a paid instance type](#) to prevent this behavior. [Learn more](#).

Logs

November 16, 2023 at 11:00 AM Live

Disks

Deploy for 586b27d 12

Environment

Search logs Search

Maximize Scroll to top

Previews

Jobs

Metrics

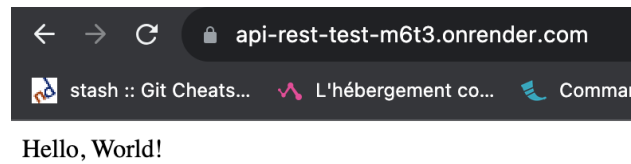
Scaling

Settings

```
Nov 16 11:00:03 AM ==> Starting service...
Nov 16 11:00:35 AM * Debug mode: off
Nov 16 11:00:35 AM WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
Nov 16 11:00:35 AM * Running on all addresses (0.0.0.0)
Nov 16 11:00:35 AM * Running on http://127.0.0.1:5001
Nov 16 11:00:35 AM * Running on http://10.217.211.93:5001
Nov 16 11:00:35 AM Press CTRL+C to quit
Nov 16 11:00:39 AM Your service is live 🎉
Nov 16 11:00:40 AM 127.0.0.1 - - [16/Nov/2023 10:00:40] "GET / HTTP/1.1" 200 -

```

Construction d'images – Déploiement



Construction d'images – Déploiement

Déploiement à partir de GitHub

Configure and deploy your new Web Service

Choose service > **2** Configure > Deploy [Need help? Docs](#)

Source Code

Git Provider **Public Git Repository** Existing Image

PR Previews and Auto-Deploy are available only for repositories configured with render.yaml

<https://github.com/render-examples/express-hello-world>

Acronyme YAML



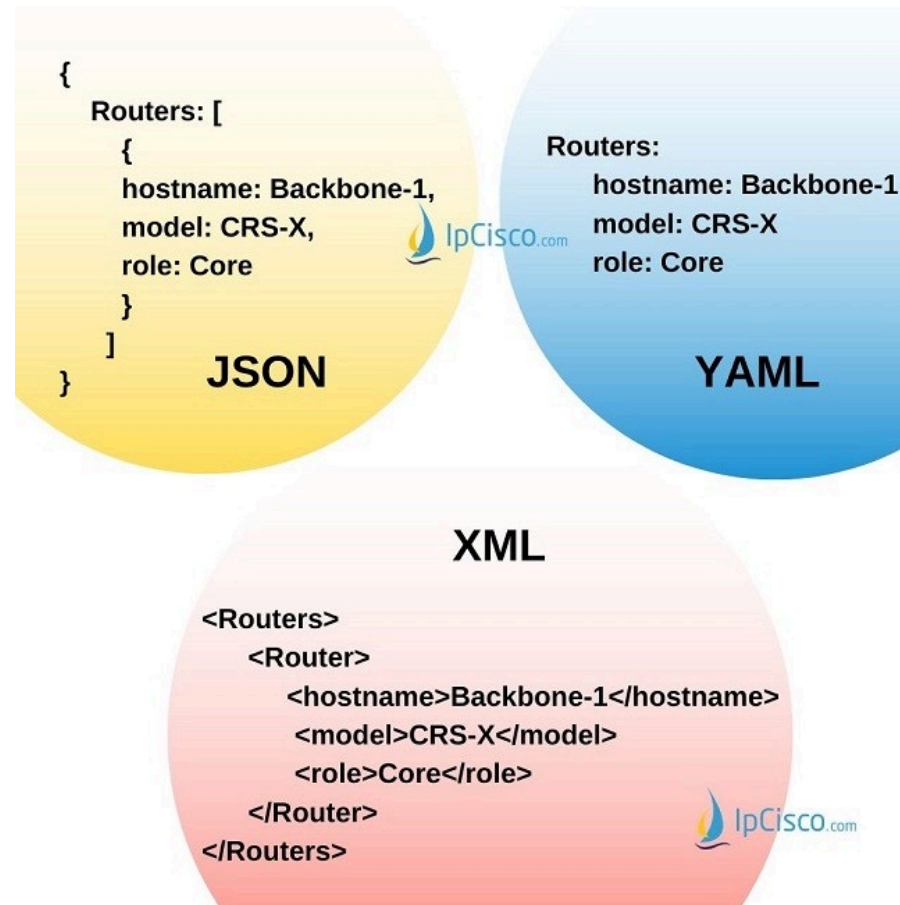
YAML Ain't Markup Language (YAML n'est pas un langage de balisage), ce qui souligne que le langage YAML s'utilise pour représenter des données plutôt que des documents.

YAML

Attribut	YAML	JSON
Verbosité	Moins verbeux	Plus verbeux
Types de données	Prend en charge les types de données complexes.	Ne prend pas en charge les types de données complexes.
commentaires	Prend en charge l'écriture de commentaires en utilisant '#'. 	Ne prend pas en charge la rédaction de commentaires.
Lisibilité	Plus lisible par l'homme.	Moins lisible par l'homme.
Auto-références	Prend en charge le référencement des éléments dans les mêmes documents en utilisant «&» et *.	Ne prend pas en charge l'auto-référencement.
Documents multiples	Prend en charge plusieurs documents dans un seul fichier.	Prend en charge un seul document dans un seul fichier.

https://fre.myservername.com/yaml-tutorial-comprehensive-guide-yaml-using-python#Benefits_Of_Using_A_YAML_File

YAML



<https://ipcisco.com/lesson/data-serialization-languages-json-yaml-xml/>

YAML

- YAML utilise une indentation de style Python pour structurer les données.
- Les caractères de tabulation n'étant pas autorisés, on utilise des espaces blancs.
- Les fichiers YAML portent l'extension .yaml ou .yml.
- Un fichier YAML est structuré sous la forme d'un mappage(dictionnaire) ou d'une liste(tableau).

Comment les données sont stockées dans YAML

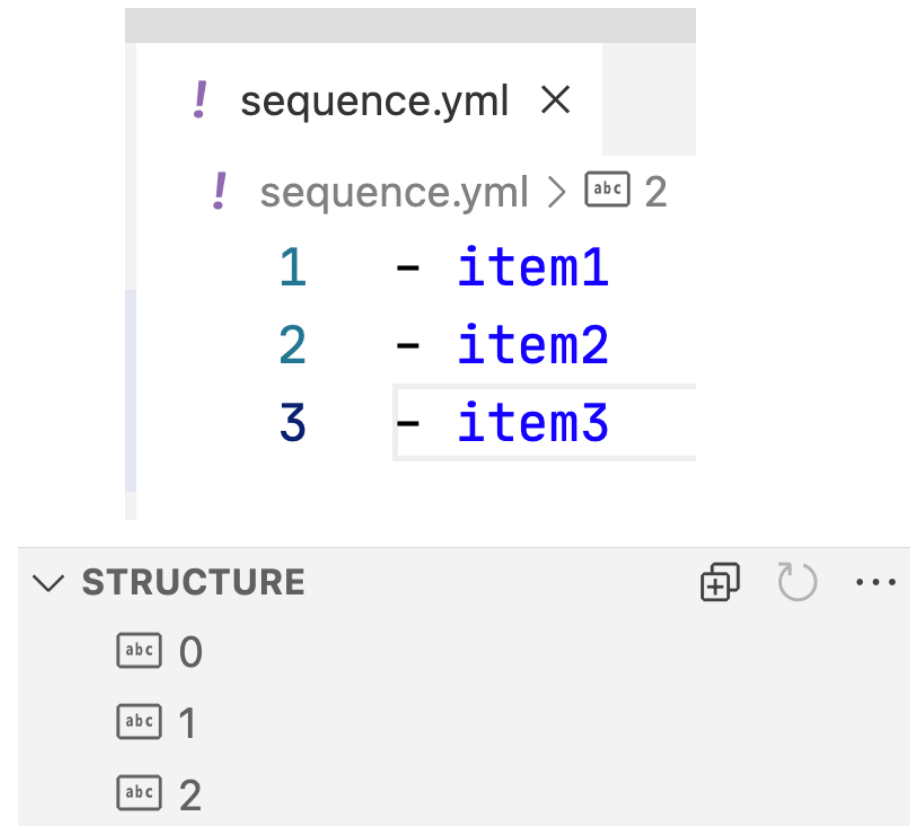
YAML peut contenir différents types de blocs de données :

- **Séquence** : valeurs répertoriées dans un ordre spécifique. Une séquence commence par un tiret et un espace (-). Vous pouvez considérer une séquence comme une **liste Python**.
- **Mapping** : paires de clés et de valeurs. Chaque clé doit être unique et l'ordre n'a pas d'importance. Pensez à un **dictionnaire Python**.

Syntaxe de base – Séquence/ tableau/ list

[item1, item2, item3]

Index	Item
0	item1
1	item2
2	item3



Syntaxe de base – Mapping/ dictionnaire/ clé-valeur

Key	Value
version	1.2
format	YAML
list	

→

java
javascript
pyhton

```
! mapping.yml ×
! mapping.yml > [ ] list > abc 2
1  version: 1.2
2  format: YAML
3  list:
4    - java
5    - javascript
6    - pyhton
```

STRUCTURE

- # version 1.2
- format YAML
- [] list
 - 0
 - 1
 - 2

Scalaire

Type de données :

- Chaîne de caractères
- Nombre
- Date
- Booléen

! scalar.yml > ...

```
1 float: 1.5
2 name: toto
3 hexadecimal: 0xABC
4
5 boolean: true
6 vide: null
7 date: 2022-11-12
```

STRUCTURE

float 1.5

abc name toto

hexadecimal 2748

boolean true

vide null

abc date 2022-11-12

Document

Plusieurs documents dans un fichier.

```
! documents.yml > ...
1  language: yaml
2  version: 1.2
3  ---
4  languages:
5  |   - java
6  |   - javascript
7  ---
8  number: 5
```

Multi-lignes

YAML ✓

```
1 multi-line-feed: |
2   line1
3   line2
4   line3
5
6 multi-line: >
7   line1
8   line2
9   line3
```

JSON

```
1 {
2   "multi-line-feed": "line1\nline2\nline3\n",
3   "multi-line": "line1 line2 line3"
4 }
```

Séquences imbriquées

! nestedSequ.yml > [] 1 > abc 2

```
1 -  
2 | - java  
3 | - javascript  
4 | - python  
5 -  
6 | - campus technique  
7 | - isims  
8 | - departement des sciences et technologies
```

STRUCTURE

✓ [] 0

abc 0

abc 1

abc 2

✓ [] 1

abc 0

abc 1

abc 2

Mappages imbriquées

! nestedMap.yml > {} person > {} address >  city

```
1  person:
2    name: toto
3    age: 20
4    address:
5      streetname: Av. V. Maistriau
6      number: 8
7      zipcode: 7000
8    city: Mons
```

✓ STRUCTURE

✓ {} person

 name toto

age 20

✓ {} address

 streetname Av. V. Maistriau

number 8

zipcode 7000

 city Mons

Combinaison mappages - séquence

```
! combMapSeq.yml > [ ] persons > {} 1 > {} address > # number
```

```
1  persons:
2    - name: toto
3      age: 20
4      address:
5        streetname: Av.V. Maistriau
6        number: 8
7    - name: titi
8      age: 30
9      address:
10     streetname: Av.V. Maistriau
11     number: 12
```

▼ STRUCTURE

```
▼ [ ] persons
  ▼ {} 0
    abc name toto
    # age 20
  ▼ {} address
    abc streetname Av.V. Maistriau
    # number 8
  ▼ {} 1
    abc name titi
    # age 30
  ▼ {} address
    abc streetname Av.V. Maistriau
    # number 12
```

Template

```
! template.yml > {} config_prod
```

```
1  config: &db #template
2    database_name: dbname
3    password: 1234test
4
5  config_prod: *db
```

✓ STRUCTURE

✓ {} config

☐ database_name dbname

☐ password 1234test

✓ {} config_prod

☐ database_name dbname

☐ password 1234test

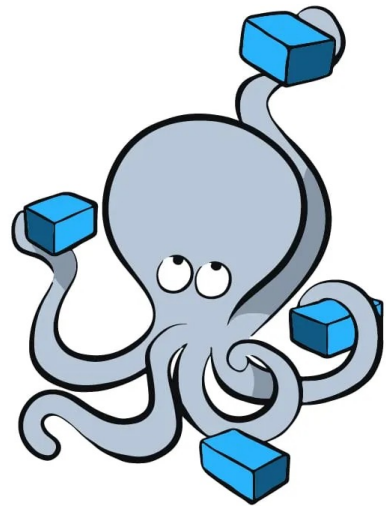
Template

YAML ✓

```
1 config: &db #template
2   database_name: dbname
3   password: 1234test
4
5 config_prod:
6   << : *db
7   password: 1234prod
```

JSON

```
1 {
2   "config": {
3     "database_name": "dbname",
4     "password": "1234test"
5   },
6   "config_prod": {
7     "database_name": "dbname",
8     "password": "1234prod"
9   }
10 }
```



docker
Compose

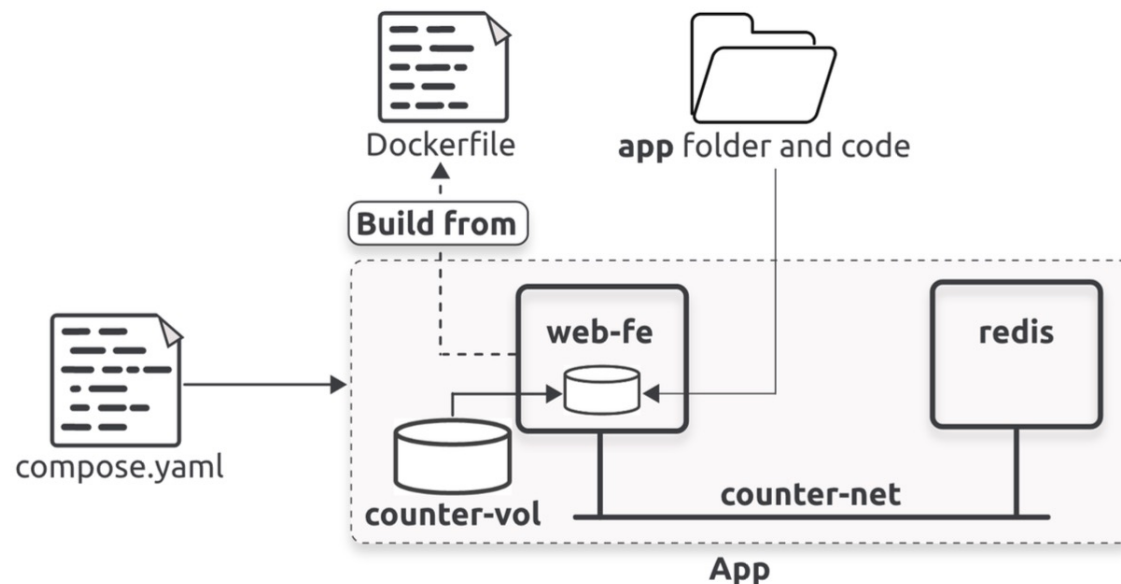
Docker compose

Docker Compose est un outil qui simplifie le **déploiement et la gestion d'applications multi-conteneurs**.

Utilité principale

- **Définir** dans un seul fichier **YAML**(`docker-compose.yml`) la configuration de plusieurs services (API, base de données, reverse proxy, cache...).
- **Automatiser** le lancement de tous les conteneurs avec une seule commande (`docker-compose up`).
- **Orchestrer** les dépendances entre services (exemple : attendre que la base de données démarre avant l'API).
- **Faciliter la portabilité** : le fichier YAML peut être partagé avec l'équipe, et tout le monde peut recréer l'environnement rapidement.
- **Simplifier le réseau** : Compose crée un réseau virtuel commun où les conteneurs communiquent par nom de service (ex : `db:3306` au lieu de `localhost:3306`).

Docker compose - Exemple



Le service **web-fe** exécute un serveur web qui incrémente un compteur dans le service **redis** chaque fois qu'il reçoit une requête pour la page web. Les deux services sont connectés au réseau **counter-net** et l'utilisent pour communiquer entre eux. Le service **web-fe** monte le volume **counter-vol**.

Docker compose - Exemple

 compose.yaml > {} services > {} redis

docker-compose.yml - '

1 **networks:**

2 | **counter-net:**

3 **volumes:**

4 | **counter-vol:**

} Ce bloc définit un nouveau réseau appelé « counter-net ».

} Ce bloc définit un nouveau volume appelé « counter-vol ».

Docker compose - Exemple

services:

▷ Run Service

```
web-fe:  ← Nom du service. Les conteneurs hériteront de ce nom.
  build: .  ← Indique que l'image Docker doit être construite à partir du Dockerfile présent dans le
             répertoire courant
  deploy:
    replicas: 1  ← Déployer un seul conteneur pour ce service
  command: python app/app.py  ← Redéfinit la commande d'exécution du conteneur.
  ports:
    - target: 8080  ← Port interne du conteneur.
      published: 5001  ← Port exposé par la machine hôte (localhost:5001).
  networks:
    - counter-net  ← Indique sur quel réseau Docker le service est connecté.
```

Docker compose - Exemple

 Dockerfile > ...

```
1  # Base image
2  FROM python:alpine (last pushed 1 month ago)
3
4  # Copy app, templates and views
5  COPY . /app
6
7  # Set working directory for subsequent commands
8  WORKDIR /app
9
10 # Install dependencies
11 RUN pip install -r requirements.txt
12
13 EXPOSE 8080
14
15 # Command to run when container starts
16 ENTRYPOINT ["python", "app/app.py"]
```

Docker compose - Exemple

```
redis:
```

```
  image: redis:alpine
```

← Pull de l'image « redis:alpine » pour ce service.

```
  deploy:
```

```
    replicas: 1
```

```
  networks:
```

```
    counter-net:
```

← Fixer les conteneurs au réseau « counter-net ».

```
  volumes:
```

```
    - type: volume
```

```
      source: counter-vol
```

```
      target: /data
```

Permet de **persister les données Redis** même si le conteneur est supprimé.

- `type: volume` => il s'agit d'un volume géré par Docker.
- `source: counter-vol` => nom du volume déclaré dans `volumes:`
- `target: /data` => dossier dans le conteneur Redis où sont stockées les données.

Docker compose - Commandes

`docker compose up` : est la commande permettant de déployer une application Compose. Elle crée toutes les images, conteneurs, réseaux et volumes dont l'application a besoin. Elle attend que vous appeliez le fichier `docker-compose.yaml`, mais vous pouvez spécifier un nom de fichier personnalisé avec le drapeau `-f`. Vous démarrez normalement l'application en arrière-plan avec le drapeau `--detach`.

`docker compose stop` : arrête tous les conteneurs d'une application Compose sans les supprimer du système.

`docker compose restart` : redémarre une application Compose arrêtée.

`docker compose ps` répertorie chaque conteneur de l'application Compose. Il affiche l'état actuel, la commande exécutée par chaque conteneur et les ports réseau.

`docker compose down` arrête et supprime une application Compose en cours d'exécution. Par défaut, il supprime les conteneurs et les réseaux, mais pas les volumes et les images.

Docker compose - Commandes

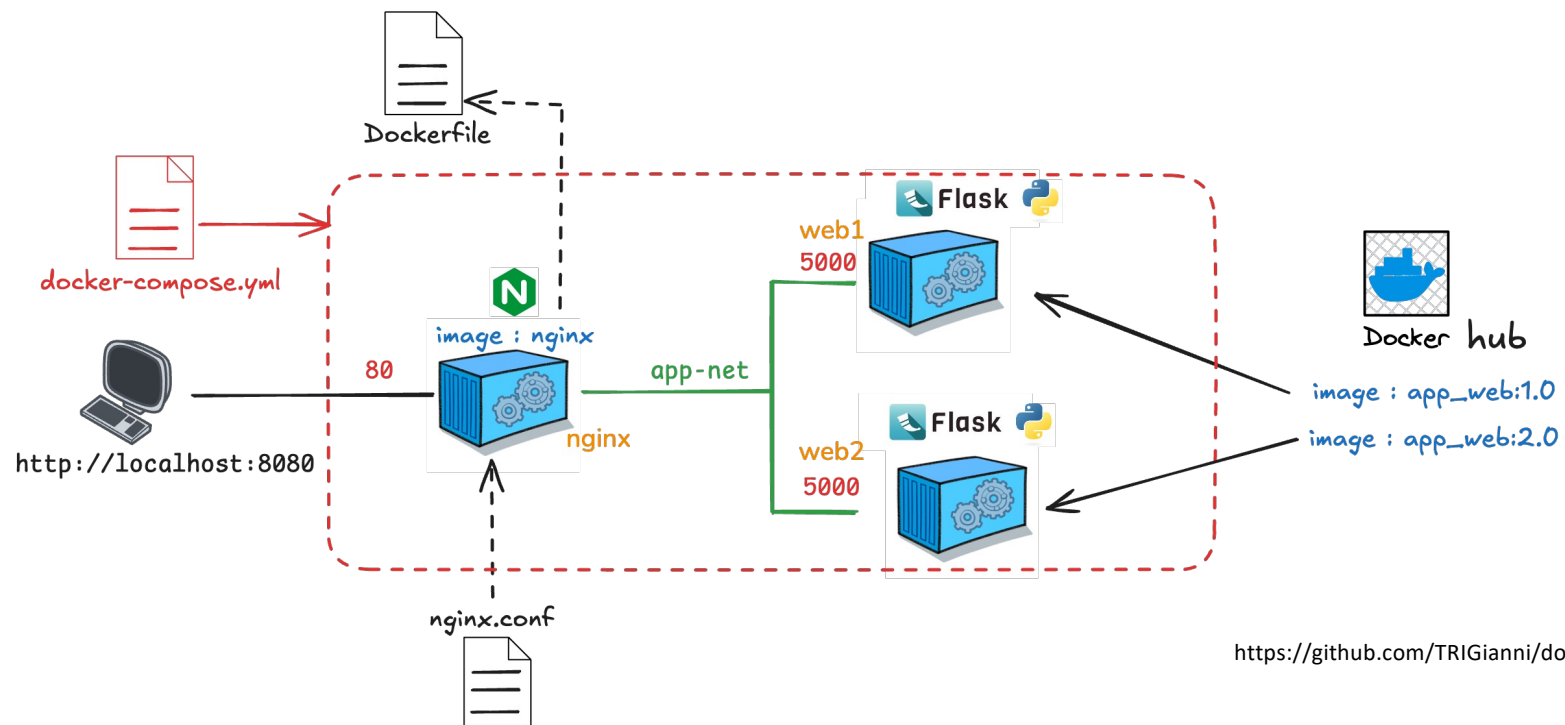
```
docker-compose down --volumes --rmi all
```

- Le drapeau `--volumes` **supprimera** tous les volumes de l'application
- Le drapeau `--rmi all` supprimera toutes ses images.

Docker compose - Exercice

Load Balancing avec Docker Compose et Nginx

Mettre en place une architecture simple composée de **deux serveurs web** derrière un **reverse proxy Nginx** qui distribue la charge entre eux (load balancing).



https://github.com/TRIGianni/dockercompose_LB

Docker compose - Exercice

Créer deux serveurs web :

- Chaque serveur doit répondre sur le port 80 interne avec un message différent (ex. `Serveur 1` et `Serveur 2`).
- Les services seront appelés `web1` et `web2`.

Créer un conteneur Nginx :

- Le service `nginx` doit agir comme un **reverse proxy** et faire du **load balancing** entre `web1` et `web2`.
- Fournir un fichier `nginx.conf` avec une directive `upstream` listant les deux serveurs backend.
- Nginx doit écouter sur le port **8080** exposé sur l'hôte.

Définir un réseau Docker :

- Tous les services doivent être connectés à un réseau personnalisé `app-net`.
- Cela permettra à Nginx d'appeler `web1` et `web2` par leur nom de service.

Docker compose - Exercice

Tester le load balancing :

- Accéder à `http://localhost:8080`.
- Rafraîchir plusieurs fois : la réponse doit alterner entre `Serveur 1` et `Serveur 2`